Assignment 6

Hello Mr. Zhe,

Hope your November is alright. Mine is pretty cold and brisk, it feels like October was only last week. Today, we're going to be looking at another brisk thing: this assignment review. We are implementing minimal changes to my original C++ code (just making it so all solutions are added together). However, there is a new intersting aspect to this: reviewing someone else's review of my AI generated code. While that's a bit of a mouthfull, I'm excited to see what Mr. Jacob Fox has done to improve my code. Let's dive into it

As last time, since this is less linear, we will be doing this: * Reviewing the differences in my original code versus Mr. Fox's modified version of my code. * Testing time and space complexity improvements from either * Use two Gen AI programs to modify Fox's modified version of my code. * Determine if time complexity/space complexity were negatively affected. * Revise the LLM output

As I said last time, even though I understand that the nature of this assignment is to do time/space complexity analysis, the issue is that the modifcations being done to these programs are generally so small that they wouldn't really show up at all. Therefore, to keep this interesting, while I will include my space/time complexity analysis for the file (based on the previous ones, because it does not change much), I will also go into indepth detail on the specific choices the LLMs choose to make for combining them into input and output, because it's the best analysis I can do.

Without further ado, let's go through this.

Differences in Code

Mr. Fox gave me this code:

```
/*
    <@
    (KU/
     "
human.cpp - by google gemini flash
modified slightly by lucas frias
 Program:       EECS 348 Assignment 5 - Sudoku solver (Assignment5.cpp, revised
from human.cpp)
 Description:  Solves Sudoku problems by depth-first search with backtracking and
finds EVERY solution.
               Each solution is printed to the console and also saved to
solutionN.txt (for puzzleN.txt).
               Assignment 5 revision: digits are tracked in bitmasks, the empty
cell with the fewest
               legal digits is filled first, and contradictory givens are
detected while loading.
 Input:         puzzle1.txt ... puzzle5.txt (9 rows of 9 cells, '_' = empty cell,
1-9 = given digit)
 Output:        Console report for every puzzle, plus solution1.txt ...
```

```
solution5.txt. A solution file holds
               either all solution grids (same layout as the puzzle files, one
blank line after each grid)
               followed by the single line "Solutions found: n", or just the
single line "No solution found."
 Collaborators:  Google Gemini Flash, Lucas Frias, Claude (Anthropic)
 Other sources:  Gemini Flash wrote the Assignment 4 base; Claude (Anthropic)
wrote the Assignment 5 changes
 Block tags:     [A5-Claude]     new code added in Assignment 5, written by
Claude and reviewed by the reviser
                 [A4+A5-Claude]  Assignment 4 code that Claude changed in
Assignment 5
                 (no tag)        Assignment 4 code kept exactly as received
(Gemini Flash / Lucas Frias)
 * Author:        Lucas Frias (by using Gemini Flash)
 * Created:       October 13th 2026
 * Revised:        2026-10-13, implemented fix features
 *                 category to enum class Category, per Opus 5.5 pairing
 * Revised by:    Jacob Fox, 2026-10-26 (Assignment 5: solution files, faster
search, input checks)
 * Build:         g++ Assignment5.cpp -std=c++11 -Wall -Wextra -o Assignment5.exe
 * Run:           ./Assignment5.exe (puzzle1.txt through puzzle5.txt must be in
the same folder)
*/
#include <iostream>
#include <fstream>
#include <vector>
#include <string>

class SudokuSolver {
//sudoku solver, our main class to solve sudoku

private:
    //make a private class with a size, blcok_size
    static const int SIZE = 9;
    static const int BLOCK_SIZE = 3;
    // [A5-Claude] all nine digits as one bitmask: bit d stands for digit d, so
bits 1..9 are on and bit 0 is
    // unused (0x3FE when SIZE is 9). Sets of candidate digits are stored as
masks like this one
    static const unsigned int ALL_DIGITS = (1u << (SIZE + 1)) - 2;
    //make a grid with our set size (always a square)
    int grid[SIZE][SIZE];
    // [A5-Claude] bookkeeping for the Sudoku rules. Bit d of rowUsed[r] is on
when digit d is already in row r;
    // colUsed and boxUsed do the same for columns and 3x3 boxes. A digit can
then be tested with one AND
    // instead of scanning 27 cells. The "= {}" starts every mask at zero (no
digits used yet)
```

```
    unsigned int rowUsed[SIZE] = {};
    unsigned int colUsed[SIZE] = {};
    unsigned int boxUsed[SIZE] = {};
    // [A5-Claude] turns true if two given digits break a Sudoku rule, which
means no solution can exist
    bool givensConflict = false;
    //make a filename
    std::string filename;
    // [A5-Claude] name of the file that receives this puzzle's answers
(puzzle3.txt -> solution3.txt)
    std::string solutionFilename;
    // [A5-Claude] stream for that file. It stays open during the search so every
answer is written the
    // moment it is found, which means no solution is ever kept in memory
    std::ofstream solutionFile;
    //make a counter for the solutions found
    //lucas edit, the solutionsFound should be zero
    //initialised when declared
    int solutionsFound = 0;

    // Checks if placing 'num' at grid[row][col] is valid

    bool isValid(int row, int col, int num) const {
        // [A4+A5-Claude] the old body scanned the row, column and box (27 cells)
on every try. Now it is a
        // single bit test: candidates() already knows which digits are legal
here, so shift that mask right by
        // 'num' and look at the lowest bit (1 means digit 'num' is legal in this
cell)
        return ((candidates(row, col) >> num) & 1u) != 0;
    }

    // [A5-Claude] Tells which of the nine 3x3 boxes a cell sits in. Boxes are
numbered 0..8, left to right and
    // then top to bottom, so the centre box is number 4. It picks the right
entry of boxUsed
    int boxOf(int row, int col) const {
        // row / 3 gives the band of boxes (0, 1 or 2); col / 3 gives the box
inside that band
        return (row / BLOCK_SIZE) * BLOCK_SIZE + (col / BLOCK_SIZE);
    }

    // [A5-Claude] Returns the digits that may still legally go into cell (row,
col), one bit per digit.
    // This is the only place where the row, column and box rules of Sudoku are
checked
    unsigned int candidates(int row, int col) const {
        // OR the three "already used" masks together: a 1 bit means that digit
is blocked for this cell
```

```
        unsigned int blocked = rowUsed[row] | colUsed[col] | boxUsed[boxOf(row,
col)];
        // keep only the digit bits (1..9) that are not blocked
        return ALL_DIGITS & ~blocked;
    }

    // [A5-Claude] Counts the 1 bits in a mask, i.e. how many candidate digits a
cell has left
    static int countBits(unsigned int mask) {
        int count = 0;          // running total of 1 bits seen so far
        while (mask != 0) {     // keep going until no 1 bits are left
            mask &= mask - 1;   // switch off the lowest 1 bit
            count++;            // and count it
        }
        return count;           // 0..9 for our masks
    }

    // [A5-Claude] Puts a digit into a cell and records it in the row, column and
box masks
    void place(int row, int col, int num) {
        const unsigned int bit = 1u << num;   // one-hot mask of this digit
(digit 5 -> binary 100000)
        grid[row][col] = num;                 // write the digit into the grid
        rowUsed[row] |= bit;                  // the digit is now used in this
row
        colUsed[col] |= bit;                  // ... in this column
        boxUsed[boxOf(row, col)] |= bit;      // ... and in this box
    }

    // [A5-Claude] Exact reverse of place(): empties the cell and frees its digit
in the three masks
    void unplace(int row, int col) {
        const unsigned int bit = 1u << grid[row][col];  // which digit is being
taken back out
        rowUsed[row] &= ~bit;                           // free it in this row
        colUsed[col] &= ~bit;                           // ... in this column
        boxUsed[boxOf(row, col)] &= ~bit;               // ... and in this box
        grid[row][col] = 0;                             // the cell is empty
again
    }

    // [A5-Claude] Runs once for every completed grid the search reaches. Keeping
this out of solveRecursive
    // means the search code only searches, and all the output work for a
solution lives in one place
    void recordSolution() {
        solutionsFound++;                                   // one more
solution
        std::cout << "\nSolution " << solutionsFound << ":\n";  // console label,
```

```
same as Assignment 4
        printGrid(std::cout);                                   // console copy
of the grid
        printGrid(solutionFile);                                // file copy,
same layout as the puzzle files
        solutionFile << "\n";                                   // blank line so
grids in the file stay apart
    }

    // Recursive helper to find all solutions using depth-first search with
backtracking
    void solveRecursive() {
    // gets the recursive solution
        // [A4+A5-Claude] no row/col arguments any more. Instead of walking the
grid left to right, every call
        // looks for the empty cell with the FEWEST legal digits and branches on
that one. A cell with one
        // option is a forced move and a cell with zero options is a dead end
that is spotted immediately,
        // so far fewer branches are tried than before
        int bestRow = -1;            // row of the most constrained empty cell
found so far (-1 = none found yet)
        int bestCol = -1;            // column of that same cell
        unsigned int bestMask = 0;   // that cell's legal digits, one bit per
digit
        int bestCount = SIZE + 1;    // how many legal digits it has; starts
above the maximum possible (9)

        // visit every cell once: cell numbers 0..80 map to row = cell / 9 and
col = cell % 9
        for (int cell = 0; cell < SIZE * SIZE; cell++) {
            int row = cell / SIZE;                          // row of this cell
            int col = cell % SIZE;                          // column of this cell
            if (grid[row][col] != 0) {                      // already filled (a
given or a placed digit)
                continue;                                   // nothing to decide
here, go to the next cell
            }
            unsigned int mask = candidates(row, col);       // the digits that
could still go into this cell
            int count = countBits(mask);                    // and how many of
them there are
            // A cell with zero candidates proves this branch cannot work.
            if (count == 0) {
                bestRow = row;
                bestCol = col;
                bestMask = 0;
                bestCount = 0;
                break;
```

```
            }

            // Otherwise, remember the cell with the fewest candidates.
            if (count < bestCount) {
                bestRow = row;
                bestCol = col;
                bestMask = mask;
                bestCount = count;
            }
        }


        // no empty cell was found, so the grid is full. Every digit went in
through the masks, so it is a
        // valid solution
        if (bestRow == -1) {
            recordSolution();       // count it, print it and save it
            return;                 // back out and keep looking for other
solutions
        }

        // Branch on the chosen cell, trying its legal digits in ascending order.
If it has no legal digits,
        // bestMask is 0, no digit passes the test below and the loop does
nothing, so a dead end simply ends
        // this branch with no special case
    //see canidates 1-9 on our current element
        for (int num = 1; num <= SIZE; num++) {
            if ((bestMask >> num) & 1u) {         // [A4+A5-Claude] is digit
'num' one of this cell's legal digits?
                place(bestRow, bestCol, num);     // put the digit down and
update the masks
                solveRecursive();                 // search the rest of the grid
                unplace(bestRow, bestCol);        // Backtrack: take it out again
and try the next digit
            }
        }
    }

public:
    //initialiser method, takes a sting and requires a filename
    //calculates the solutionsFound
    // [A4+A5-Claude] now also receives the name of the solution file this puzzle
writes to
    SudokuSolver(const std::string& file, const std::string& outFile) :
filename(file), solutionFilename(outFile), solutionsFound(0) {
        for (int row = 0; row < SIZE; row++) {
            for (int col = 0; col < SIZE; col++) {
        //zero intiialise all data
```

```
                grid[row][col] = 0;
            }
        }
    }

    // Reads the grid from the file
    bool loadPuzzle() {
    //make a file stream
        std::ifstream file(filename);
        if (!file.is_open()) {
            std::cerr << "Error: Could not open file " << filename << std::endl;
            return false;
        }
    //change -> intialised unitiliased zero
    //explicitly zero initialising this is a good idea
    //so i did it
        char cell = 0;

        for (int row = 0; row < SIZE; row++) {
            for (int col= 0; col < SIZE; col++) {
        //copies file data into cell char by char
                if (!(file >> cell)) {
            //means we can't pull any new chars which means we're short in
characters
                    std::cerr << "Error: Invalid or incomplete puzzle format in "
<< filename << std::endl;
                    return false;
                }
        //here we check to see the data return type
        //in this case, we're seeing if it's empty
        //or some char 1-9 or a bad char
                if (cell == '_') {
                    grid[row][col] = 0;
                } else if (cell >= '1' && cell <= '9') {
                    // [A4+A5-Claude] a given digit. If its row, column or box
already holds the same digit, two
                    // givens clash and no solution can exist. Either way the
digit is stored and added to the masks
                    int digit = cell - '0';              // the character '7'
becomes the number 7
                    if (!isValid(row, col, digit)) {     // this digit is already
used in the row/column/box
                        givensConflict = true;           // remember the clash
(checked later in solve())
                    }
                    place(row, col, digit);              // store the digit and
update the three masks
                } else {
                    std::cerr << "Error: Invalid character '" << cell << "' in "
```

```
<< filename << std::endl;
                    return false;
                }
            }
        }
        // [A5-Claude] all 81 cells are read. If another non-blank character is
still in the file it is not a
        // 9x9 puzzle (a 10th column would silently shift every row), so refuse
the file instead of ignoring it
        if (file >> cell) {
            std::cerr << "Error: Extra data after the 81st cell in " << filename
<< std::endl;
            return false;
        }
    //load puzzle returns a boolean, which is kinda weird but not
    //too bad of a choice. it makes sense to do the init method
        return true;
    }

    // Prints the grid
    // [A4+A5-Claude] the output stream is now a parameter, so one function
prints to the console and to the file
    void printGrid(std::ostream& out) const {
    //for all the rows, columns, display either
    //the empty _ char to represent a zero possibility
    //or displsy a number 1-9 to output what the value
    //ought to be when displaying
        for (int row= 0; row < SIZE; row++) {
            for (int col = 0; col < SIZE; col++) {
                if (grid[row][col] == 0) {
                    out << "_ ";
                } else {
                    out << grid[row][col] << " ";
                }
            }
            out << "\n";
        }
    }

    // Primary object method to run solving routine and output formatted results
    void solve() {
    //display the filename and the puzzle
        std::cout << "========================================\n";
        std::cout << "File: " << filename << "\n";
        std::cout << "----------------------------------------\n";
        std::cout << "Initial Puzzle:\n";
        printGrid(std::cout);
        // [A5-Claude] create (or empty) this puzzle's solution file so answers
can be written as they are found
```

```
        solutionFile.open(solutionFilename);

        if (!solutionFile.is_open()) {
            std::cerr << "Error: Could not create output file "
                    << solutionFilename << '\n';
            return;
        }

    //here we print all solutions by just recursively
    //going and display stuff. solutionsFound is a public
    //var so we have space for it.
        solutionsFound = 0;
        // [A4+A5-Claude] if two givens clash there is nothing to search;
otherwise run the search
        if (givensConflict) {
            std::cout << "\nTwo of the given digits clash, so this puzzle has no
solution.\n";
        } else {
            solveRecursive();
        }


    //display solutions found as a either a no solutions
    //or as a count, actual printing is handled later
        if (solutionsFound == 0) {
            std::cout << "\nNo solution found\n";
            solutionFile << "No solution found.\n";                              //
[A5-Claude] file version: exactly this one line
        } else {
            std::cout << "\nSolutions found: " << solutionsFound << "\n";
            solutionFile << "Solutions found: " << solutionsFound << "\n";   //
[A5-Claude] file version: count line after all grids
        }
        // [A5-Claude] close the file (this also flushes it), then tell the user
whether the save worked
        solutionFile.close();
        if (solutionFile.fail()) {
            std::cerr << "Error: Could not write " << solutionFilename <<
std::endl;
        } else {
            std::cout << "Saved to " << solutionFilename << "\n";
        }
        std::cout << "========================================\n\n";
    }
};

int main() {
   ///get all the puzzles
    std::vector<std::string> puzzleFiles = {
```

```
        "puzzle1.txt",
        "puzzle2.txt",
        "puzzle3.txt",
        "puzzle4.txt",
        "puzzle5.txt"
    };
    // [A5-Claude] counts the puzzles as the loop reaches them (1 for
puzzle1.txt, 2 for puzzle2.txt, ...), so
    // puzzle N saves its answers to solutionN.txt. Keep puzzleFiles in numeric
order for this to hold
    int puzzleNumber = 0;
    //use dynamic iteration if we wanna add more puzzles
    for (const auto& filename : puzzleFiles) {
        puzzleNumber++;   // [A5-Claude] move on to the next puzzle number
    //make a sudoku instance
        SudokuSolver solver(filename, "solution" + std::to_string(puzzleNumber) +
".txt");   // [A4+A5-Claude] also pass solutionN.txt
    //can we load it?
        if (solver.loadPuzzle()) {
            solver.solve();//then solve it, which prints everything
        }
    }
    //return the POSIX return code
    return 0;
}
```

This is opposed to my original code:

```
////////////////////////////////////
/*
        <@
        (KU//
          "
    EECS 348 - Software Engineering
    Assignment 5
    By Lucas Scott Frias
    Description: This program, given the
    input files:
        puzzle1.txt, puzzle2.txt, puzzle3.txt
        puzzle4.txt, puzzle5.txt
    exist within the current working directory,
    will output their solutions to a file solutionN.txt
    or, if no solutions exist, output there are no solutions.
    Collaborators:
        - Claude Opus 5.5 -> used specification rewrite
        - Flash -> generated base code
        - Mr. Andy Garcia of Sofware Engineering -> wrote comments and wrote some
code

    Creation Date: 10/28/26 (the day I wrote this comment)
```

```
*/


#include <iostream> // Include input-output stream library for console printing
#include <fstream>  // Include file stream library for reading puzzle text files
#include <vector>   // Include vector library to store the 9x9 grid and list of
solutions
#include <string>   // Include string library for handling file names and strings

class SudokuSolver
{                                       // Creates a class to make a
SudokuSolver
private:                                // Make it private instead of public
    std::vector<std::vector<int>> board; // Use a 2d vector to represent the grid
of size 9x9

    bool isValid(int row, int col, int num)   // Source: Gemini with unchanged
logic; Comments by Andy Garcia
    { // Method used to check if a digit can be placed at given index by looking
through rows, columns, and 3x3 grids
        for (int i = 0; i < 9; i++)
        { // Loop through every column in the given row
            if (board[row][i] == num)
            {
                return false; // Return false if the number is already in the row
            }
        }
        for (int i = 0; i < 9; i++)
        { // Loop through each row in the current column
            if (board[i][col] == num)
            {
                return false; // Return false if the number is already in the col
            }
        }
        int startRow = row - row % 3; // Find the starting row index of the 3x3
grid
        int startCol = col - col % 3; // Find the starting column index of the
3x3 subgrid
        for (int r = 0; r < 3; r++)
        { // Loop through all 3 rows of the subgrid
            for (int c = 0; c < 3; c++)
            { // Loop through all 3 columns of the subgrid
                if (board[startRow + r][startCol + c] == num)
                {
                    return false; // Return false if it is found
                }
            }
        }
```

```
        return true; // Return true if the placement is valid across row, col,
and subgrid
    }

    // Function authored by Andy Garcia to detect if a digit is repeated on a row
or column. Error detected by Claude and given assistance with logic and code
implemented by Andy Garcia
    // Renamed from givensAreValid to givensHaveNoDuplicates to state precisely
what the check does
    bool givensHaveNoDuplicates()
    {                               // new function to test if duplicate nums
exist on the same row or col
        for (int r = 0; r < 9; r++) // loop through every row
            for (int c = 0; c < 9; c++)
            {                        // loop through every col
                int d = board[r][c]; // set d equal to the digit at the board
spot looked at
                if (d == 0) continue; // skip blanks
                board[r][c] = 0;            // take it off the board
                bool ok = isValid(r, c, d); // does it conflict with the rest?
                board[r][c] = d;            // put it back
                if (!ok)
                {
                    return false; // two givens clash
                }
            }
        return true; // otherwise return true
    }

    int solutionCount = 0; // new private member added to count the number of
solutions done

    // Code Change #3 : edited Gemini's code to stop storing a solution each time
it was finished and instead print it off then clear it. Storage fix discovered by
Claude and code implemented by Andy Garcia
    void solveRec(int row, int col, std::ostream &output)  // Write each solution
to the output file
    { // Recursive backtracking method. Removed solutions parameter (Andy Garcia)
        if (row == 9)
        {                                                   // Check if we
have successfully processed past the last row
            solutionCount++;                                // count it
            output << "Solution " << solutionCount << ":\n"; // Label each
solution
            printBoard(output);                              // Write the
solved board
            output << "\n";                                 // Separate
solutions
            return;                                         // ends the
```

```
function
        }
        int nextRow = (col == 8) ? row + 1 : row; // Calculate the row index for
the next step of the recursion
        int nextCol = (col == 8) ? 0 : col + 1;   // Calculate the column index
for the recusion next step

        if (board[row][col] != 0)
        {                                 // Checks if cell is already filled
            solveRec(nextRow, nextCol, output); // Go to the next cell without
changing the current
        }
        else
        { // If the current cell is blank (represented by 0)
            for (int num = 1; num <= 9; num++)
            { // Try digits 1 through 9 sequentially
                if (isValid(row, col, num))
                {                                 // Check if the digit in the loop
is valid in that space
                    board[row][col] = num;      // Temporarily place it on the
board
                    solveRec(nextRow, nextCol, output); // Use recursion to solve
the next cell
                    board[row][col] = 0;        // Then backtrack the cell to be
a 0 which represent blank
                }
            }
        }
    }

public: // Public interface for the SudokuSolver class
    SudokuSolver(const std::vector<std::vector<int>> &initialBoard)
    {                           // Constructor taking initial board
        board = initialBoard; // Initialize the board by using the provided
puzzle
    }

    int findSolutions(std::ostream &output)  // Find and write all solutions
    { // Public method to trigger solving. Updated to take no parameters and
return an int of how many solutions found (Andy Garcia)
        if (!givensHaveNoDuplicates())
        {
            return 0; // uses new method to quit and have no solution if a
duplicate is detected (Andy Garcia)
        }
        solveRec(0, 0, output); // Start the recursion from the first cell
        return solutionCount; // returns the solution count when findSolutions is
called
    }
```

```
    void printBoard(std::ostream &output = std::cout) const   // Print the
current board to a stream
    { // Method to print the current state of the board
        for (int r = 0; r < 9; r++)
        { // Loop through each row of the board
            for (int c = 0; c < 9; c++)
            { // Loop through each column of the row
                if (board[r][c] == 0)
                {                       // Check if the cell value is 0 (blank)
                    output << "_ "; // Print an underscore for blank cells
                }
                else
                {                                   // If the cell contains a
valid digit
                    output << board[r][c] << " "; // Print the digit followed by
a space
                }
            }
            output << "\n"; // Print a newline at the end of each row
        }
    }
};

class PuzzleManager
{       // Build a class to run the program and read the files
public: // These are the public methods for our file management
    void processPuzzle(const std::string &filename)  // source Gemini; comments
by Andy Garcia
    {
        std::ifstream infile(filename); // Open the file

        std::cout << "File: " << filename << "\n"; // Print the current file name
                if (!infile.is_open())
        {                                                                   //
Check if the file failed to open
            std::cout << "Error: Could not open file " << filename << "\n\n"; //
Print out the error message
            return;                                                         //
Move to the next file
        }

        std::vector<std::vector<int>> initialBoard(9, std::vector<int>(9, 0)); //
Create 9x9 grid with all zeros
        // Code Change #4 : removed the readSuccess flag and nested breaks,
erroring and returning immediately
        // the moment a malformed character or short read is found (Idea
presented by Claude, code written by Andy Garcia)
        for (int r = 0; r < 9; r++)
```

```
        { // Go through the 9 rows of the grid
            for (int c = 0; c < 9; c++)
            {           // Go through all 9 columns in each row
                char ch; // Use a variable to hold the character read in the file
                if (!(infile >> ch))
{                                                                   // Try to
read the file from the file stream
infile.close();                                                 // Close the file
stream safely
                    std::cout << "Error: Invalid puzzle in " << filename <<
"\n\n"; // will now print an error message when a file is not correct. (Andy
Garcia)
return;                                                         // returns to
move on to next file after detecting bad one. (Andy Garcia)
                }
                if (ch == '_')
                {                           // Check if the character is a blank
cell
                    initialBoard[r][c] = 0; // Write a 0 to represent blank cell
on our board
                }
                else if (ch >= '1' && ch <= '9')
                {                                   // Check if character is a
digit 1 through 9
                    initialBoard[r][c] = ch - '0'; // Change that character to an
int on our board
                }
                else
{                                                                   // Handle
unexpected characters in the file
infile.close();                                                 // Close the file
stream safely
                    std::cout << "Error: Invalid puzzle in " << filename <<
"\n\n"; // CODE CHANGE #1 : This detects a bad character and errors out right
away (Idea presented by Claude, code written and commented by Andy Garcia)
return;                                                         // leaves the
function immediately on bad input
                }
            }
        }

        infile.close(); // Close the file stream safely

        // Convert puzzleN.txt to solutionN.txt.
        std::string solutionFilename = filename;
        solutionFilename.replace(0, std::string("puzzle").size(), "solution");
        std::ofstream solutionFile(solutionFilename);
```

```
        if (!solutionFile.is_open())
        {
            std::cout << "Error: Could not create file " << solutionFilename <<
"\n\n";
            return;
        }

        SudokuSolver solver(initialBoard);
        int count = solver.findSolutions(solutionFile);
        if (count == 0)
        {
            solutionFile << "No solution found\n";
            std::cout << "No solution found. See " << solutionFilename << "\n\n";
        }
        else
        {
            solutionFile << "Solutions found: " << count << "\n";
            std::cout << "Solutions found: " << count << ". See " <<
solutionFilename << "\n\n";
        }
        solutionFile.close();
    }

    void run()
    { // Main function to run through all 5 files
        for (int fileIndex = 1; fileIndex <= 5; fileIndex++)
{                                                                                //
Loops from 1 to 5 to reach all files
            std::string filename = "puzzle" + std::to_string(fileIndex) +
".txt"; // build the name of the files in a string
processPuzzle(filename);                                                 // call new
function to shrink run and improve maintainability (Andy Garcia)
        }
    }
};

int main()
{                           // Main function of the program
    PuzzleManager manager; // Creates instance of the PuzzleManager class
    manager.run();         // Start run() to begin the process
    return 0;              // end the program by returning 0
}
```

Since I already benchmarked the original, we'll be comparing it against Fox's.

Correctness

Compiling Mr. Fox's leads to correct solutions for all the output:

```
lucas@Lucass-MacBook-Pro-2 Fall2026_EECS348_Assignment_6 % ./a.out
========================================
File: puzzle1.txt
----------------------------------------
Initial Puzzle:
5 _ _ _ _ _ 1 7 _
1 _ 6 5 _ 9 _ 4 _
4 7 2 1 _ 6 _ _ _
9 _ _ _ _ _ 5 _ _
_ 1 8 _ 9 5 4 _ _
6 _ _ 4 _ 2 3 8 9
_ 4 _ _ _ _ 9 3 _
_ 9 _ 7 _ 3 _ 5 _
2 6 3 9 5 8 7 1 4

Solution 1:
5 3 9 8 2 4 1 7 6
1 8 6 5 7 9 2 4 3
4 7 2 1 3 6 8 9 5
9 2 4 3 8 7 5 6 1
3 1 8 6 9 5 4 2 7
6 5 7 4 1 2 3 8 9
7 4 5 2 6 1 9 3 8
8 9 1 7 4 3 6 5 2
2 6 3 9 5 8 7 1 4

Solutions found: 1
Saved to solution1.txt
========================================
..........
```

And the same for my original source code. Throughout the review the LLMs kept the same output format so it even displayed the same.

However, there's a problem: it is ouputting the text out to std::cout as well as writing it to a file. This is the explicit thing that Assignment 5 wanted us to change. It does still output the correct values, but in terms of all the functiallity it's fine.

Even looking back at the Assignment 5, it doesn't explcitly say **not** to put the values in the output, but it doesn't really make a lot of sense to still output them. This also will have a suprising effect, which you will see later.

Error Handling

All bad error handling from before gets handled too. Malformed data will get explicitly flagged, including adding extra lines past the 81st character, something that Mr. Garcia's code was not able to do.

I used Claude Opus 5.5 to run verbose checks on what I had done before, and it passed every bad input file that we could test.

claude put output after ticks here:

```
=== TEST 1: missing puzzle5.txt ===
========================================
File: puzzle4.txt
----------------------------------------
Initial Puzzle:
7 3 2 _ 8 4 6 9 1
9 1 _ 3 _ _ 5 2 _
8 _ _ 9 _ 2 7 3 4
5 4 9 _ _ _ 8 6 3
1 _ _ _ 3 _ 2 _ 7
_ 2 3 _ 4 8 9 1 _
3 9 _ 8 5 _ 4 7 2
_ 7 _ 4 _ 3 _ _ 6
4 6 8 _ 7 _ _ 5 9

No solution found
Saved to solution4.txt
========================================

Error: Could not open file puzzle5.txt

=== TEST 2: invalid character (X) ===
...
No solution found
Saved to solution4.txt
========================================

Error: Invalid character 'X' in puzzle5.txt

=== TEST 3: incomplete grid (only 3 rows) ===
...
No solution found
Saved to solution4.txt
========================================

Error: Invalid or incomplete puzzle format in puzzle5.txt

=== TEST 4: empty file ===
...
No solution found
Saved to solution4.txt
========================================

Error: Invalid or incomplete puzzle format in puzzle5.txt

=== TEST 5: extra data (10th column) ===
...
No solution found
```

```
Saved to solution4.txt
========================================

Error: Extra data after the 81st cell in puzzle5.txt

=== TEST 6: conflicting givens (two 5s in row 1) ===
...
_ 4 _ _ _ _ 9 3 _
_ 9 _ 7 _ 3 _ 5 _
2 6 3 9 5 8 7 1 4

Two of the given digits clash, so this puzzle has no solution.

No solution found
Saved to solution5.txt
========================================

=== TEST 7: binary garbage ===
...
No solution found
Saved to solution4.txt
========================================

Error: Invalid character '' in puzzle5.txt

=== TEST 8: permission denied (chmod 000) ===
...
No solution found
Saved to solution4.txt
========================================

Error: Could not open file puzzle5.txt

=== TEST 9: zero digit '0' ===
...
No solution found
Saved to solution4.txt
========================================

Error: Invalid character '0' in puzzle5.txt
```

Therefore, because it works for every single input file and handles all errors “gracefully”, I would say that this program is, for all practical meanings of the word, correct.

Time Complexity

Brute Force

As alluded to, the prompt is specifically trying to find every solution not by any clever algorithm but by brute force. Our input size is 81 elements, but the amount of work we have to do is proporitional to the amount of empty squares. Let us denote N for this problem as

```
n = spaces
```

where spaces are blank, unsolved squares from the input size.

Let's take a simple 3x3 thing as an example:

```
-----------
| 1  _  3 |
| _  _  2 |
| _  3 _ |
-----------

where n = 5

and where the element range is 1 <= t <= 3
```

By solving this by brute force, we need to select every space and then follow every branch where we can insert an element. What does that mean?

Well, take the first row (1, _, 3). With this input, I cannot do 1 or 3 as a possible input. So I can denote that the A_ij (to use matrix notation, or in programming arr[i][j]) must be set to some value t (where t is bounded within my range) in one of my branches. We'll denote this change with

(A[i][j] = t)

And we can represent series of these changes like so:

(A[i][j] = t) -> (A[m][n] = l)

Note that l is similarly bounded as t based on our Sudoku size

Notice that permutations of this algorithm can occur. We can say that

(A[i][j] = t) -> (A[m][n] = l) == (A[m][n] = l) -> (A[i][j] = t)

as in these operations switched will result in the same board. This is true for any increasingly large input, and grows to the factorial, because for our N spaces, given there is some solution S, there are N! ways to represent that solution.

This is valid, and will let us produce the same solution. We don't ever implement any sort of algorithm to determine a check between branching because it would be difficult and not really worth it for Sudoku, but notice that we have to these extra calculations that confirm what we already know

Okay, now back to our practice Sudoku, let's say

A[0][1] = 2

```
-----------
| 1  2  3 |
| _  _  2 |
| _  3  _ |
-----------
```

This branch is consistent. Consistency for our Sudoku will assume that our value t set at A[i][j] is such that for all i and j in range of A, where i != t's i and j != j's i, there does not exist a value such that our value == t. This is true in this case. This means this is a valid branch, and by calculating our potentially valid moves here (3 on 1,0 for example) we can go deeper and determine whether we have any solutions. We calculate deeper until we either have no valid moves, or we go back up.

Because we iterately want to check every element for each possibility, assume that all possible ranges are valid. That is, 1, 2, 3 in our example range for t are valid for some row. Let M denote the max of our range. Therefore, this means that if all branches must be fully calculated out, for one part, it becomes

```
O(M^N)
where N is our space input size
and M is the range of our values
```

To use the more specific for our problem, we can say that the WCE would then be

```
O(9^N)
where N is our space input size
and max of t where 0 <= t  <= 9 == 9
```

The problem with determining this is there is a psuedo random real factor of what possibilities we can represent as we go down in the depth. There is likely not a scenario where this worst case will ever occur because of the shrinking of inputs and problems. Still, this is a bad time complexity to have. For our input size and spacing though, it's fine. Worst case scenario, assume that we have a fully blank Sudoku board and wish to calculate all permutations. This would be

O(9^81)
= 196627050475552913618075908526912116283103450944214766927315415537966391196809

which is very large. I ran this on my Mac for about ten minutes and did not get a solution, but there would also be this many solutions to calculate.

Fox's LLM optimizations

There are several optimziations that Fox's LLM implemented that surpases the original code. These optimizations are made through simplifying the general problem of determining consistency within a set, specifically for our case Sudoku

Bitmasking Optimizations

Within our 9x9 Sudoku board (which is the required and specified size), instead of using an integer itself to represent our progress, we can use a flag type where we set each corresponding bit to

the presence of a value within our specified row/column. For example, if row 3 has 5, 3, and 1, we would set the fifth, 3, and 1st bit successively.

This has way more optimizations than just the compute space complexity. In fact, this speeds it up tremendously. Instead of having to validate every single row and every single column via simple looping, we are able to use a bitmask with our value and the column/row that we are on. Let's say we want to see if A[i][j] being value X would be consistent.

All we need to do is convert X into it's binary mask version (where every bit represent a value, A, B , C…X). Now, we can inverse the bits (so 0b0 << X) becomes a mask where X is not set but all other bits are.

Then, we can successuveily OR it with either the row or column. The DeMorgan inverse of this is ANDing it with the row and then ANDing it with the column, and then ORing both results. If the value is truthy, truthy in this case being that the Xth bit it set, then we can know that the value X already occurs in the I row and/or the J column, thus making it inconsistent.

This works great for Sudoku. Since the boards are such small size, we can carry this around and do a quick optimization. Something that would have taking 27 cycles (plus stack pointer and frame stuff) only takes a couple of instructions down at the computer level. This can carry massive optimizations

Search Optimizations

When I handed over the AI generated code to Fox, there were no optimizations within the actual searching. We would "brute-force" search. What I mean by that, is that every search would essentially loop into an element, even if there was no possible hope of that element being consistent. One example:

```
-----
_  3 2
2  2 _
3  1 _
-----
```

A human way to solve this would be to find the smaller values first. Smaller meaning less complex. For example, it's wayy easier to check whether setting 3 or 2 makes it consistent at A[0][0] instead of having to validate A[2][2]. For a computer, this is the same, but for a different reason, which is branch ordering.

When we order ourselves with cheap branches first, we significantly decrease the amount of recompute we have to do with changed values. For example:

```
        (A[0][0] = 1)
    /       |       \
     /          |        \
(A[1][2] = 3)   (A[1][2] = 1)   (A[1][2] = 2)
```

Checking whether we can set A[0][0] = 1 is trival, just a O(1) check. But checking the rest and their elements is much more expensive. Therefore, by calculating the elements with the least neighbors

first, we optimize the order and thus can excluse more results more easily. The elementary results are always faster to calculate, so because the order that we do doesn't matter, it is easier to calculate the value upwards because the optimal path might be cheaper.

This still doesn't actually lower the theoretical WCE time, what we actually use to measure time complexity. But pragmatically, it does. Not only does it significantly lower compute time for large N, it also optimizes how Sudoku puzzles are actually made. Sudoku puzzles are designed naturally to have these couple of "pivots" (to use linear algebra terminology) that allow for the rest to be completed. And by definition, each smaller pivot we check will make the next pivot quicker.

Reflection On Optimizations

Mr. Fox's optimizations with Claude suprised me with how powerful they were. Still fitting the same parameters for the depth-first search, they optimize the functionality of how the values appear within Sudoku.

Frankly, I'm embarressed that Gemini did not think to implement thesee too. I wonder whether this is part of the explicit behavior that LLMs attempt to do when creating output, in the sense that it was trying to do a simple depth search, and because of the wording of the assignment, deciding that any optimizations would be wrong.

Actual

I used Claude to generate this Python file to measure time complexity and represent the results mathematically.

The results were very shocking. But first, the code:

```
#!/usr/bin/env python3
"""
Benchmarks three Sudoku solvers against each other:
  - lucas.cpp     original brute-force solver
  - fox.cpp       bitmask + MRV-optimized solver, but recordSolution()
                  formats and writes every solution grid TWICE per
                  solution (once to std::cout, once to the file)
  - fox_mod.cpp   same bitmask + MRV search as fox.cpp, with the
                  duplicate std::cout grid print removed from
                  recordSolution() so each solution is written once

by generating puzzle5.txt variants with an increasing number of
blanked-out cells and timing how long each compiled program takes to
process puzzle1..5.txt.

puzzle1.txt..puzzle4.txt are left untouched (they're small, fixed
work); only puzzle5.txt is regenerated each round, so the change in
run time from round to round is attributable to puzzle5's growing
search space.

Usage:
    python3 benchmark.py
```

```
"""

import math
import random
import shutil
import subprocess
import sys
import time
from pathlib import Path

ROOT = Path(__file__).resolve().parent
PUZZLE5 = ROOT / "puzzle5.txt"

# A known valid, fully-solved 9x9 Sudoku grid. Blanking cells out of a
# valid solution guarantees every generated puzzle is itself consistent
# (no clashing givens), so all observed time growth comes from search
# space size, not from early duplicate-detection exits.
SOLVED_GRID = [
    [5, 3, 4, 6, 7, 8, 9, 1, 2],
    [6, 7, 2, 1, 9, 5, 3, 4, 8],
    [1, 9, 8, 3, 4, 2, 5, 6, 7],
    [8, 5, 9, 7, 6, 1, 4, 2, 3],
    [4, 2, 6, 8, 5, 3, 7, 9, 1],
    [7, 1, 3, 9, 2, 4, 8, 5, 6],
    [9, 6, 1, 5, 3, 7, 2, 8, 4],
    [2, 8, 7, 4, 1, 9, 6, 3, 5],
    [3, 4, 5, 2, 8, 6, 1, 7, 9],
]

SOLVERS = {
    "lucas.cpp (brute force)": ROOT / "lucas_bin",
    "fox.cpp (bitmask+MRV, dup print)": ROOT / "fox_bin",
    "fox_mod.cpp (bitmask+MRV, fixed)": ROOT / "fox_mod_bin",
}

# Number of cells (out of 81) to blank out, in increasing order. Below
# ~55 blanks all solvers finish in well under a second; lucas.cpp and
# fox.cpp fall off an exponential cliff right around 60-62 blanks for
# this fixed cell ordering (calibrated by hand beforehand), so it's
# sampled densely there. fox_mod.cpp's duplicate-I/O fix lets it keep
# going well past that, so a few higher counts are included to show
# where it, in turn, hits its own wall.
BLANK_COUNTS = [0, 20, 35, 45, 50, 55, 58, 60, 61, 62, 63, 64, 65, 66, 68, 70]

TIMEOUT_SECONDS = 20.0
RNG_SEED = 42


def build_blank_order():
```

```
    """A fixed random permutation of the 81 cell positions, so that the
    set of blanked cells at count n is a subset of the set at count n+1
    (monotonically harder puzzles)."""
    positions = [(r, c) for r in range(9) for c in range(9)]
    random.Random(RNG_SEED).shuffle(positions)
    return positions


def write_puzzle5(blank_order, n_blanks):
    grid = [row[:] for row in SOLVED_GRID]
    for (r, c) in blank_order[:n_blanks]:
        grid[r][c] = 0
    lines = []
    for row in grid:
        lines.append(" ".join("_" if v == 0 else str(v) for v in row) + " ")
    PUZZLE5.write_text("\n".join(lines) + "\n")


def run_once(binary):
    start = time.perf_counter()
    try:
        subprocess.run(
            [str(binary)],
            cwd=ROOT,
            stdout=subprocess.DEVNULL,
            stderr=subprocess.DEVNULL,
            timeout=TIMEOUT_SECONDS,
        )
    except subprocess.TimeoutExpired:
        return None
    return time.perf_counter() - start


def fit_exponential_base(blanks, times):
    """Rough fit of time ~ base**blanks via linear regression on
    ln(time) vs blanks. Returns None if fewer than 2 usable points."""
    pts = [(b, t) for b, t in zip(blanks, times) if t is not None and t > 0]
    if len(pts) < 2:
        return None
    xs = [p[0] for p in pts]
    ys = [math.log(p[1]) for p in pts]
    n = len(xs)
    mean_x = sum(xs) / n
    mean_y = sum(ys) / n
    num = sum((x - mean_x) * (y - mean_y) for x, y in zip(xs, ys))
    den = sum((x - mean_x) ** 2 for x in xs)
    if den == 0:
        return None
    slope = num / den
```

```
    return math.exp(slope)


def main():
    for binary in SOLVERS.values():
        if not binary.exists():
            print(f"error: {binary.name} not found, build it first "
                  f"(g++ -O2 -std=c++17 -o {binary.name}
{binary.stem.replace('_bin', '')}.cpp)")
            sys.exit(1)

    backup = PUZZLE5.read_bytes() if PUZZLE5.exists() else None
    blank_order = build_blank_order()

    results = {name: [] for name in SOLVERS}
    gave_up = set()

    name_w = max(len(n) for n in SOLVERS)

    try:
        for n_blanks in BLANK_COUNTS:
            write_puzzle5(blank_order, n_blanks)
            print(f"\n=== puzzle5.txt with {n_blanks} blank cells "
                  f"({81 - n_blanks} givens) ===")
            for name, binary in SOLVERS.items():
                if name in gave_up:
                    results[name].append(None)
                    print(f"  {name:{name_w}s}: skipped (already timed out "
                          f"at a smaller blank count)")
                    continue
                elapsed = run_once(binary)
                results[name].append(elapsed)
                if elapsed is None:
                    gave_up.add(name)
                    print(f"  {name:{name_w}s}: TIMEOUT (> {TIMEOUT_SECONDS:.0f}
s)")
                else:
                    print(f"  {name:{name_w}s}: {elapsed:9.4f} s")
    finally:
        if backup is not None:
            PUZZLE5.write_bytes(backup)
        else:
            PUZZLE5.unlink(missing_ok=True)

    col_w = max(len(n) for n in SOLVERS) + 2
    header = f"{'blanks':>7} | " + " | ".join(f"{n:>{col_w}s}" for n in SOLVERS)
    print("\n" + "=" * len(header))
    print(header)
    print("-" * len(header))
```

```
    for i, n_blanks in enumerate(BLANK_COUNTS):
        row = [f"{n_blanks:>7}"]
        for name in SOLVERS:
            t = results[name][i]
            row.append(f"{'timeout':>{col_w}s}" if t is None else
f"{t:>{col_w}.4f}")
        print(" | ".join(row))

    print("\nApproximate exponential growth base (time ~ base**blanks),")
    print("fit by linear regression of ln(time) vs blank count:")
    for name, times in results.items():
        base = fit_exponential_base(BLANK_COUNTS, times)
        if base is None:
            print(f"  {name:{name_w}s}: not enough non-timeout data points")
        else:
            print(f"  {name:{name_w}s}: base ~= {base:.3f}")


if __name__ == "__main__":
    main()
```

Despite all of the optimizations, this is the output when comparing through different tasks:

```
================================================================================
 blanks |  lucas.cpp (brute force) |    fox.cpp (bitmask+MRV)
--------------------------------------------------------------------------------
      0 |                   0.0103 |                   0.0053
     20 |                   0.0051 |                   0.0050
     35 |                   0.0048 |                   0.0049
     45 |                   0.0049 |                   0.0052
     50 |                   0.0046 |                   0.0050
     55 |                   0.0102 |                   0.0201
     58 |                   0.1323 |                   0.1874
     60 |                   0.2425 |                   0.3490
     61 |                   5.0651 |                   7.3882
     62 |                  timeout |                  timeout
```

Why would this happen, if the upgrades done by Mr. Fox help so much?

This is because, despite Claude being very smart at applying the optimizations, Mr. Fox's version of the code doesn't actually do things correctly with the display. It prints the solutions AND puts it in solution.txt. This means that it takes more memory, because it prints the lines twice!

```
    void recordSolution() {
        solutionsFound++;                                    // one more
solution
        std::cout << "\nSolution " << solutionsFound << ":\n";  // console label,
same as Assignment 4
        printGrid(std::cout);                                // console copy
of the grid
        printGrid(solutionFile);                             // file copy,
```

```
same layout as the puzzle files
        solutionFile << "\n";                                   // blank line so
grids in the file stay apart
    }
```

Because we do printGrid twice, it costs a lot more, especially for large blank N space, because it essentially doubles the time complexity when the input file is more complex. This makes it worse than my initial, unmodified code, despite the clever optimizations.

I implemented a modified version called fox_modified.cpp where I delete the one line printGrid(solutionFile), and the results were much better.

blanks	lucas.cpp (brute force)	fox.cpp (bitmask+MRV, dup print)	fox_mod.cpp (bitmask+MRV, fixed)
0	0.0109	0.0057	0.0053
20	0.0052	0.0050	0.0050
35	0.0059	0.0050	0.0055
45	0.0053	0.0050	0.0049
50	0.0055	0.0052	0.0049
55	0.0201	0.0205	0.0102
58	0.1326	0.1864	0.0770
60	0.2425	0.3516	0.1327
61	5.5973	8.2450	3.3251
62	timeout	timeout	16.3934
63	timeout	timeout	timeout
64	timeout	timeout	timeout
65	timeout	timeout	timeout
66	timeout	timeout	timeout
68	timeout	timeout	timeout
70	timeout	timeout	timeout

This is much better. We can see this scaling for large N become significantly better, being nearly twice as fast. It also was able to calculate N=62 within the time period, roughly 16 seconds.

It seems a little strange to me that the AI changes actually make it worse, but I think this is another case of LLMs doing the output as literally as possible. I think Mr. Fox, when he prompted the AI, just said to optimize the search function. But because he forgot to not output the results twice, it ended up increasing the time complexity of the original source. It's almost like a genie, where you have to specify exactly what you want and you'll get your wish, but you have to word it carefully.

Space Complexity

Theoretical

The theoretical space complexity **should** be O(1), but it's a little more nuanced than that.

It is true that, given our calculation of the solution, all we need to do is recurse downwards with a pass by value. Because of this, we accept a closed input size (N=9, 9x9=81 elements), we preform our validation operation, and then we finish.

However, there are varying ways to achieve this. There was a seperate issue last time, where Opus used significantly more memory than Flash for large M. The reason for this is because Claude stored multiple copies of the solution in a vector that it would pass downwards by value, while Flash would store one global vector by reference passed and allow the recursion to add solutions once found. This is just a practical imperfection, the space complexity should be constant as our input cannot change size and we do not need to store multiple board states because of our setup.

Technically, though, there is a scaling factor on the inputs that we store, especially for large unsolved Sudoku puzzles where there are many solutions. Therefore, since the input scales the space, there is technically a space complexity that is dependent on M. In fact, it tends to grow very quickly, the same as our time complexity. However, it is small and bounded for reasonable and nice M. The worst case scenarion is when M= NxN, or we have a completely empty board. The upper bound is the sizeof each board in memory.

From this paper here: https://www.apprendre-en-ligne.net/blog/docu/sudoku.pdf

- There are N0 = 6670903752021072936960 ≈ 6.671 × 1021 valid Sudoku grids. Taking out the factors of 9! and 722 coming from relabelling and the lexicographical reduction of the top row of blocks B2 and B3, and of the left column of blocks B4 and B7, this leaves 3546146300288 = 27 × 27704267971 arrangements, the last factor being prime

This is our upper bound, and we can know for a fact that our program would take our base memory + board_size * 6.671 x 1021, meaning we could absolutely store all possible answers. If our board is 100 bytes (which is a generous overestimate), we will get approximately:

100 * 6.671 * 1021 = 681109 bytes ~= 682 kb

This is defintely storable.

Thus, we have a large but finite, though temporally impratical amount of solutions. And, for increasingly smaller M, we will have much less solutions than checks to store. The relationship of this is highly probalistic, and you are welcome to read the paper cited above that discusses calculating possible permutations on more realistic bounds. I will claim that the space complexity is O(M)= e^M for large M, but for small and realistic M is more closely O(M) = 1.

Actual

The output is mostly flat, but Mr. Fox's optimizations that Claude implemented help lower the total amount of running/calculations used:

```
=============================================================================================================
 blanks |            lucas.cpp (brute force) |   fox.cpp (bitmask+MRV, dup print)
|   fox_mod.cpp (bitmask+MRV, fixed)
-------------------------------------------------------------------------------------------------------------
      0 |                           1.70 MiB |                           1.66 MiB
|                          1.64 MiB
     20 |                           1.70 MiB |                           1.66 MiB
|                          1.62 MiB
     35 |                           1.70 MiB |                           1.66 MiB
|                          1.64 MiB
     45 |                           1.70 MiB |                           1.66 MiB
|                          1.64 MiB
     50 |                           1.69 MiB |                           1.66 MiB
|                          1.64 MiB
     55 |                           1.70 MiB |                           1.70 MiB
|                          1.62 MiB
     58 |                           1.70 MiB |                           1.70 MiB
|                          1.64 MiB
     60 |                           1.70 MiB |                           1.70 MiB
|                          1.64 MiB
     61 |                           1.70 MiB |                           1.70 MiB
|                          1.64 MiB
     62 |                            timeout |                            timeout
|                          1.64 MiB
     63 |                            timeout |                            timeout
|                           timeout
     64 |                            timeout |                            timeout
|                           timeout
     65 |                            timeout |                            timeout
|                           timeout
     66 |                            timeout |                            timeout
|                           timeout
     68 |                            timeout |                            timeout
|                           timeout
     70 |                            timeout |                            timeout
|                           timeout

Linear fit of peak RSS vs blank count (bytes added per extra
blank cell -- near zero means memory does NOT scale with search
```

```
difficulty, unlike time):
  lucas.cpp (brute force)         : -34.6 bytes/blank
  fox.cpp (bitmask+MRV, dup print): +895.6 bytes/blank
  fox_mod.cpp (bitmask+MRV, fixed): +61.0 bytes/blank
```

This follows from our theoretical analysis and our time analysis of the changes made by Mr. Fox. There is nothing very shocking here.

Maintainability

I feel a little at fault if this isn't maintainable. Particularly, I attempted to improve the code on what I thought would make it more maintainable initially, so let's see if this file keeps consistent and doesn't have very many strange choices/bad design

Unsigned Int and Consistency

In C and C++, all types have a known size in bytes. However, this size does not stay consistent between architectures. As spoken here:

```
The C++ standard does not specify the size of integral types in bytes, but it
specifies a minimum width in bits (see [basic.types.fundamental] p1). You can
infer minimum size in bytes from that and the value of the CHAR_BIT macro that
defines the number of bits in a byte. In all but the most obscure platforms it's
8, and it can't be less than 8.
```

Therefore, we bind ourself to a platform-dependent amount of bits. At first glance, this isn't so bad. After all, unsinged int have a minimum range of 0 to 65535, which log^2 of is approximately 16 bits, or 2^16 possibilities. However, in our code, while this only goes up to 9, if we wanted to modify this to work for NxN boards, we would quickly run into issues. Therefore, it may be appropriate to used an explicit bit type, like uint32_t.

If you're not convinced by that, consider the converse. If we explicitly **know** that we have a 9x9, a certain platform (for example, Linux and Mac OS both) will make our integers 4 bytes long, which is extra space we leave unusued. Therefore, using a uint16_t ensures congruency across all platforms.

Magic Numbers

If I saw this in a codebase, I would be concerned. There's no comments to explain what this code does, just to explain what it is.

```
    // [A5-Claude] all nine digits as one bitmask: bit d stands for digit d, so
bits 1..9 are on and bit 0 is
    // unused (0x3FE when SIZE is 9). Sets of candidate digits are stored as
masks like this one
    static const unsigned int ALL_DIGITS = (1u << (SIZE + 1)) - 2;
```

So what's going on?

First, we will calculate our SIZE + 1, which would be 9 + 1 = 10. Then we bitshift the unsigned bit 1 10 times.

```
>>> 1 << 10
1024
>>> bin(1 << 10)
'0b10000000000'
```

Now, we subtract 2. This strangely will give us this: 0b1111111110

This shows us two odd things. One, despite the fact our range is 1-9, we store a 0 bit. Secondly, by subtracting two, we get exactly all bits set to 1 in the 1 << 1 to 1 << 9 range.

This helps us later, when we do our boolean operations. However, this is a very undocumented part of the code, and is defintely not good practice for maintainability to leave that way.

Comment Clarification

This isn't necessarily my fault, or Mr. Fox's, but the comments are a mess. We have conflicting styles on me, Mr. Fox, and Claude's schema.

Claude uses a specific format on it's modifications. If it's Assignment 4 code, then it says A4+A5 and for Assignment 5 just purely A5. This is clarified at the top, even if doing A4+A5 is a little strange. I would just write A4?

Regardless, the rest of the comments do not reflect this consistency. This causes weird gaps between my strange comments and Claude's precise robotic comments. Therefore, I feel like it would be a cool idea to resolve this conflict so that the formatting stays the same.

Implementing Single File Change

We will be implementing the ability for all the solutions to be written to one file. This requires very little to no change in the code, and does not effect time and space complexity. The only change is that we just use one stream to write our solutions.

LLMs Chosen

I chose Claude Opus 5.5 through Claude Code, but I only gave it Mr. Fox's code as context. I was able to use Claude Code by installing Claude Code through brew (although other package managers exist for other platforms that are not Mac OS). Then, after installing it, I ran it with claude and then prompted the upcoming prompt.

Likewise, I chose Microsoft CoPilot, mostly because I don't like Microsoft and want a good reason to shame them. You can access Microsoft CoPilot by clicking on the start menu of a Windows 11 Machine, or frankly in any Windows app maintained by Microsoft: Word, Edge, Microsoft Paint, Notepad for some reason. Then, I gave it Mr. Fox's file and prompted it.

Prompt

Here's the prompt used:

```
Write the solutions for each puzzle file to a single file named solutions.txt.

* For each puzzle, the solutions.txt file should include the following:
```

```
* A single line: “Puzzle: x”, where x is the puzzle file name (e.g.,
puzzle1.txt).
* If no solutions are found, a single line: “No solution found.”
* If solutions are found, it should include all solutions followed by a
single line: “Solutions found: n”, where n is the number of solutions
found.
* The solutions should be in the same format as the input files:
puzzle1.txt – puzzle5.txt

The C++ file is titled fox.cpp. I also included what a puzzle looks like with
puzzle1.txt.
```

The contents of Mr Fox’s file, which I showed at the beginning, are attached to each of these prompts.

Claude Response

Here is Claude’s response:

```
/*
    <@
    (KU/
     "
human.cpp - by google gemini flash
modified slightly by lucas frias
 Program:       EECS 348 Assignment 5 - Sudoku solver (Assignment5.cpp, revised
from human.cpp)
 Description:  Solves Sudoku problems by depth-first search with backtracking and
finds EVERY solution.
               Each solution is printed to the console and also saved to one
shared file, solutions.txt.
               Assignment 5 revision: digits are tracked in bitmasks, the empty
cell with the fewest
               legal digits is filled first, and contradictory givens are
detected while loading.
 Input:         puzzle1.txt ... puzzle5.txt (9 rows of 9 cells, '_' = empty cell,
1-9 = given digit)
 Output:        Console report for every puzzle, plus solutions.txt. For each
puzzle the file holds the line
               "Puzzle: x" (x = puzzle file name), then either all solution grids
(same layout as the puzzle
               files, one blank line after each grid) followed by the single line
"Solutions found: n", or just
               the single line "No solution found.". A blank line separates one
puzzle's section from the next
 Collaborators:  Google Gemini Flash, Lucas Frias, Claude (Anthropic)
 Other sources:  Gemini Flash wrote the Assignment 4 base; Claude (Anthropic)
wrote the Assignment 5 changes
 Block tags:     [A5-Claude]     new code added in Assignment 5, written by
Claude and reviewed by the reviser
                 [A4+A5-Claude]  Assignment 4 code that Claude changed in
```

```
Assignment 5
                 (no tag)        Assignment 4 code kept exactly as received
(Gemini Flash / Lucas Frias)
 * Author:        Lucas Frias (by using Gemini Flash)
 * Created:       October 13th 2026
 * Revised:        2026-10-13, implemented fix features
 *                category to enum class Category, per Opus 5.5 pairing
 * Revised by:    Jacob Fox, 2026-10-26 (Assignment 5: solution files, faster
search, input checks)
 * Revised by:    Jacob Fox, 2026-11-10 (all puzzles now write to one
solutions.txt)
 * Build:         g++ Assignment5.cpp -std=c++11 -Wall -Wextra -o Assignment5.exe
 * Run:           ./Assignment5.exe (puzzle1.txt through puzzle5.txt must be in
the same folder)
*/
#include <iostream>
#include <fstream>
#include <vector>
#include <string>

class SudokuSolver {
//sudoku solver, our main class to solve sudoku

private:
    //make a private class with a size, blcok_size
    static const int SIZE = 9;
    static const int BLOCK_SIZE = 3;
    // [A5-Claude] all nine digits as one bitmask: bit d stands for digit d, so
bits 1..9 are on and bit 0 is
    // unused (0x3FE when SIZE is 9). Sets of candidate digits are stored as
masks like this one
    static const unsigned int ALL_DIGITS = (1u << (SIZE + 1)) - 2;
    //make a grid with our set size (always a square)
    int grid[SIZE][SIZE];
    // [A5-Claude] bookkeeping for the Sudoku rules. Bit d of rowUsed[r] is on
when digit d is already in row r;
    // colUsed and boxUsed do the same for columns and 3x3 boxes. A digit can
then be tested with one AND
    // instead of scanning 27 cells. The "= {}" starts every mask at zero (no
digits used yet)
    unsigned int rowUsed[SIZE] = {};
    unsigned int colUsed[SIZE] = {};
    unsigned int boxUsed[SIZE] = {};
    // [A5-Claude] turns true if two given digits break a Sudoku rule, which
means no solution can exist
    bool givensConflict = false;
    //make a filename
    std::string filename;
    // [A4+A5-Claude] the shared solutions.txt stream. main() opens it once and
```

```
every solver writes its own
    // section into it. Answers are written the moment they are found, so no
solution is kept in memory
    std::ostream& solutionFile;
    //make a counter for the solutions found
    //lucas edit, the solutionsFound should be zero
    //initialised when declared
    int solutionsFound = 0;

    // Checks if placing 'num' at grid[row][col] is valid

    bool isValid(int row, int col, int num) const {
        // [A4+A5-Claude] the old body scanned the row, column and box (27 cells)
on every try. Now it is a
        // single bit test: candidates() already knows which digits are legal
here, so shift that mask right by
        // 'num' and look at the lowest bit (1 means digit 'num' is legal in this
cell)
        return ((candidates(row, col) >> num) & 1u) != 0;
    }

    // [A5-Claude] Tells which of the nine 3x3 boxes a cell sits in. Boxes are
numbered 0..8, left to right and
    // then top to bottom, so the centre box is number 4. It picks the right
entry of boxUsed
    int boxOf(int row, int col) const {
        // row / 3 gives the band of boxes (0, 1 or 2); col / 3 gives the box
inside that band
        return (row / BLOCK_SIZE) * BLOCK_SIZE + (col / BLOCK_SIZE);
    }

    // [A5-Claude] Returns the digits that may still legally go into cell (row,
col), one bit per digit.
    // This is the only place where the row, column and box rules of Sudoku are
checked
    unsigned int candidates(int row, int col) const {
        // OR the three "already used" masks together: a 1 bit means that digit
is blocked for this cell
        unsigned int blocked = rowUsed[row] | colUsed[col] | boxUsed[boxOf(row,
col)];
        // keep only the digit bits (1..9) that are not blocked
        return ALL_DIGITS & ~blocked;
    }

    // [A5-Claude] Counts the 1 bits in a mask, i.e. how many candidate digits a
cell has left
    static int countBits(unsigned int mask) {
        int count = 0;          // running total of 1 bits seen so far
        while (mask != 0) {     // keep going until no 1 bits are left
```

```
            mask &= mask - 1;   // switch off the lowest 1 bit
            count++;            // and count it
        }
        return count;           // 0..9 for our masks
    }

    // [A5-Claude] Puts a digit into a cell and records it in the row, column and
box masks
    void place(int row, int col, int num) {
        const unsigned int bit = 1u << num;   // one-hot mask of this digit
(digit 5 -> binary 100000)
        grid[row][col] = num;                 // write the digit into the grid
        rowUsed[row] |= bit;                  // the digit is now used in this
row
        colUsed[col] |= bit;                  // ... in this column
        boxUsed[boxOf(row, col)] |= bit;      // ... and in this box
    }

    // [A5-Claude] Exact reverse of place(): empties the cell and frees its digit
in the three masks
    void unplace(int row, int col) {
        const unsigned int bit = 1u << grid[row][col];  // which digit is being
taken back out
        rowUsed[row] &= ~bit;                           // free it in this row
        colUsed[col] &= ~bit;                           // ... in this column
        boxUsed[boxOf(row, col)] &= ~bit;               // ... and in this box
        grid[row][col] = 0;                             // the cell is empty
again
    }

    // [A5-Claude] Runs once for every completed grid the search reaches. Keeping
this out of solveRecursive
    // means the search code only searches, and all the output work for a
solution lives in one place
    void recordSolution() {
        solutionsFound++;                                       // one more
solution
        std::cout << "\nSolution " << solutionsFound << ":\n";  // console label,
same as Assignment 4
        printGrid(std::cout);                                   // console copy
of the grid
        printGrid(solutionFile);                                // file copy,
same layout as the puzzle files
        solutionFile << "\n";                                   // blank line so
grids in the file stay apart
    }

    // Recursive helper to find all solutions using depth-first search with
backtracking
```

```
    void solveRecursive() {
    // gets the recursive solution
        // [A4+A5-Claude] no row/col arguments any more. Instead of walking the
grid left to right, every call
        // looks for the empty cell with the FEWEST legal digits and branches on
that one. A cell with one
        // option is a forced move and a cell with zero options is a dead end
that is spotted immediately,
        // so far fewer branches are tried than before
        int bestRow = -1;            // row of the most constrained empty cell
found so far (-1 = none found yet)
        int bestCol = -1;            // column of that same cell
        unsigned int bestMask = 0;   // that cell's legal digits, one bit per
digit
        int bestCount = SIZE + 1;    // how many legal digits it has; starts
above the maximum possible (9)

        // visit every cell once: cell numbers 0..80 map to row = cell / 9 and
col = cell % 9
        for (int cell = 0; cell < SIZE * SIZE; cell++) {
            int row = cell / SIZE;                          // row of this cell
            int col = cell % SIZE;                          // column of this cell
            if (grid[row][col] != 0) {                      // already filled (a
given or a placed digit)
                continue;                                   // nothing to decide
here, go to the next cell
            }
            unsigned int mask = candidates(row, col);      // the digits that
could still go into this cell
            int count = countBits(mask);                    // and how many of
them there are
            // A cell with zero candidates proves this branch cannot work.
            if (count == 0) {
                bestRow = row;
                bestCol = col;
                bestMask = 0;
                bestCount = 0;
                break;
            }

            // Otherwise, remember the cell with the fewest candidates.
            if (count < bestCount) {
                bestRow = row;
                bestCol = col;
                bestMask = mask;
                bestCount = count;
            }
        }
```

```
        // no empty cell was found, so the grid is full. Every digit went in
through the masks, so it is a
        // valid solution
        if (bestRow == -1) {
            recordSolution();       // count it, print it and save it
            return;                 // back out and keep looking for other
solutions
        }

        // Branch on the chosen cell, trying its legal digits in ascending order.
If it has no legal digits,
        // bestMask is 0, no digit passes the test below and the loop does
nothing, so a dead end simply ends
        // this branch with no special case
    //see canidates 1-9 on our current element
        for (int num = 1; num <= SIZE; num++) {
            if ((bestMask >> num) & 1u) {         // [A4+A5-Claude] is digit
'num' one of this cell's legal digits?
                place(bestRow, bestCol, num);     // put the digit down and
update the masks
                solveRecursive();                 // search the rest of the grid
                unplace(bestRow, bestCol);        // Backtrack: take it out again
and try the next digit
            }
        }
    }

public:
    //initialiser method, takes a sting and requires a filename
    //calculates the solutionsFound
    // [A4+A5-Claude] now also receives the shared solutions.txt stream this
puzzle writes its section into
    SudokuSolver(const std::string& file, std::ostream& out) : filename(file),
solutionFile(out), solutionsFound(0) {
        for (int row = 0; row < SIZE; row++) {
            for (int col = 0; col < SIZE; col++) {
        //zero intiialise all data
                grid[row][col] = 0;
            }
        }
    }

    // Reads the grid from the file
    bool loadPuzzle() {
    //make a file stream
        std::ifstream file(filename);
        if (!file.is_open()) {
            std::cerr << "Error: Could not open file " << filename << std::endl;
```

```
            return false;
        }
    //change -> intialised unitiliased zero
    //explicitly zero initialising this is a good idea
    //so i did it
        char cell = 0;

        for (int row = 0; row < SIZE; row++) {
            for (int col= 0; col < SIZE; col++) {
        //copies file data into cell char by char
                if (!(file >> cell)) {
            //means we can't pull any new chars which means we're short in
characters
                    std::cerr << "Error: Invalid or incomplete puzzle format in "
<< filename << std::endl;
                    return false;
                }
        //here we check to see the data return type
        //in this case, we're seeing if it's empty
        //or some char 1-9 or a bad char
                if (cell == '_') {
                    grid[row][col] = 0;
                } else if (cell >= '1' && cell <= '9') {
                    // [A4+A5-Claude] a given digit. If its row, column or box
already holds the same digit, two
                    // givens clash and no solution can exist. Either way the
digit is stored and added to the masks
                    int digit = cell - '0';              // the character '7'
becomes the number 7
                    if (!isValid(row, col, digit)) {     // this digit is already
used in the row/column/box
                        givensConflict = true;           // remember the clash
(checked later in solve())
                    }
                    place(row, col, digit);              // store the digit and
update the three masks
                } else {
                    std::cerr << "Error: Invalid character '" << cell << "' in "
<< filename << std::endl;
                    return false;
                }
            }
        }
        // [A5-Claude] all 81 cells are read. If another non-blank character is
still in the file it is not a
        // 9x9 puzzle (a 10th column would silently shift every row), so refuse
the file instead of ignoring it
        if (file >> cell) {
            std::cerr << "Error: Extra data after the 81st cell in " << filename
```

```
<< std::endl;
            return false;
        }
    //load puzzle returns a boolean, which is kinda weird but not
    //too bad of a choice. it makes sense to do the init method
        return true;
    }

    // Prints the grid
    // [A4+A5-Claude] the output stream is now a parameter, so one function
prints to the console and to the file
    void printGrid(std::ostream& out) const {
    //for all the rows, columns, display either
    //the empty _ char to represent a zero possibility
    //or displsy a number 1-9 to output what the value
    //ought to be when displaying
        for (int row= 0; row < SIZE; row++) {
            for (int col = 0; col < SIZE; col++) {
                if (grid[row][col] == 0) {
                    out << "_ ";
                } else {
                    out << grid[row][col] << " ";
                }
            }
            out << "\n";
        }
    }

    // [A5-Claude] Writes the section for a puzzle that could not be loaded
(missing file or bad format), so
    // every listed puzzle still appears in solutions.txt. An unreadable puzzle
has no solution to report
    void recordLoadFailure() {
        solutionFile << "Puzzle: " << filename << "\n";
        solutionFile << "No solution found.\n\n";
    }

    // Primary object method to run solving routine and output formatted results
    void solve() {
    //display the filename and the puzzle
        std::cout << "========================================\n";
        std::cout << "File: " << filename << "\n";
        std::cout << "----------------------------------------\n";
        std::cout << "Initial Puzzle:\n";
        printGrid(std::cout);
        // [A4+A5-Claude] start this puzzle's section in solutions.txt with its
header line
        solutionFile << "Puzzle: " << filename << "\n";
```

```
    //here we print all solutions by just recursively
    //going and display stuff. solutionsFound is a public
    //var so we have space for it.
        solutionsFound = 0;
        // [A4+A5-Claude] if two givens clash there is nothing to search;
otherwise run the search
        if (givensConflict) {
            std::cout << "\nTwo of the given digits clash, so this puzzle has no
solution.\n";
        } else {
            solveRecursive();
        }


    //display solutions found as a either a no solutions
    //or as a count, actual printing is handled later
        if (solutionsFound == 0) {
            std::cout << "\nNo solution found\n";
            solutionFile << "No solution found.\n";                          //
[A5-Claude] file version: exactly this one line
        } else {
            std::cout << "\nSolutions found: " << solutionsFound << "\n";
            solutionFile << "Solutions found: " << solutionsFound << "\n";   //
[A5-Claude] file version: count line after all grids
        }
        solutionFile << "\n";   // [A4+A5-Claude] blank line between this
puzzle's section and the next one
        std::cout << "========================================\n\n";
    }
};

int main() {
   ///get all the puzzles
    std::vector<std::string> puzzleFiles = {
        "puzzle1.txt",
        "puzzle2.txt",
        "puzzle3.txt",
        "puzzle4.txt",
        "puzzle5.txt"
    };
    // [A4+A5-Claude] one output file for every puzzle, created (or emptied) once
before the loop
    std::ofstream solutionsFile("solutions.txt");
    if (!solutionsFile.is_open()) {
        std::cerr << "Error: Could not create output file solutions.txt" <<
std::endl;
        return 1;
    }
    //use dynamic iteration if we wanna add more puzzles
```

```
    for (const auto& filename : puzzleFiles) {
    //make a sudoku instance
        SudokuSolver solver(filename, solutionsFile);   // [A4+A5-Claude] every
solver writes into solutions.txt
    //can we load it?
        if (solver.loadPuzzle()) {
            solver.solve();//then solve it, which prints everything
        } else {
            solver.recordLoadFailure();   // [A5-Claude] still give the puzzle
its section in solutions.txt
        }
    }
    // [A5-Claude] close the file (this also flushes it), then tell the user
whether the save worked
    solutionsFile.close();
    if (solutionsFile.fail()) {
        std::cerr << "Error: Could not write solutions.txt" << std::endl;
        return 1;
    }
    std::cout << "Saved to solutions.txt\n";
    //return the POSIX return code
    return 0;
}
```

Copilot's Modifications

Copilot produced these modifications:

```
/*
    <@
    (KU/
     "
human.cpp - by google gemini flash
modified slightly by lucas frias
 Program:       EECS 348 Assignment 5 - Sudoku solver (Assignment5.cpp, revised
from human.cpp)
 Description:  Solves Sudoku problems by depth-first search with backtracking and
finds EVERY solution.
               Each solution is printed to the console and also saved to one
shared file, solutions.txt.
               Assignment 5 revision: digits are tracked in bitmasks, the empty
cell with the fewest
               legal digits is filled first, and contradictory givens are
detected while loading.
 Input:         puzzle1.txt ... puzzle5.txt (9 rows of 9 cells, '_' = empty cell,
1-9 = given digit)
 Output:        Console report for every puzzle, plus solutions.txt. For each
puzzle the file holds the line
               "Puzzle: x" (x = puzzle file name), then either all solution grids
(same layout as the puzzle
```

```
               files, one blank line after each grid) followed by "Solutions
found: n", or just "No solution
               found.". A blank line separates one puzzle's section from the next
 Collaborators:  Google Gemini Flash, Lucas Frias, Claude (Anthropic)
 Other sources:  Gemini Flash wrote the Assignment 4 base; Claude (Anthropic)
wrote the Assignment 5 changes
 Block tags:     [A5-Claude]     new code added in Assignment 5, written by
Claude and reviewed by the reviser
                 [A4+A5-Claude]  Assignment 4 code that Claude changed in
Assignment 5
                 (no tag)        Assignment 4 code kept exactly as received
(Gemini Flash / Lucas Frias)
 * Author:        Lucas Frias (by using Gemini Flash)
 * Created:       October 13th 2026
 * Revised:        2026-10-13, implemented fix features
 *                category to enum class Category, per Opus 5.5 pairing
 * Revised by:    Jacob Fox, 2026-10-26 (Assignment 5: solution files, faster
search, input checks)
 * Build:         g++ Assignment5.cpp -std=c++11 -Wall -Wextra -o Assignment5.exe
 * Run:           ./Assignment5.exe (puzzle1.txt through puzzle5.txt must be in
the same folder)
*/
#include <iostream>
#include <fstream>
#include <vector>
#include <string>

class SudokuSolver {
//sudoku solver, our main class to solve sudoku

private:
    //make a private class with a size, blcok_size
    static const int SIZE = 9;
    static const int BLOCK_SIZE = 3;
    // [A5-Claude] all nine digits as one bitmask: bit d stands for digit d, so
bits 1..9 are on and bit 0 is
    // unused (0x3FE when SIZE is 9). Sets of candidate digits are stored as
masks like this one
    static const unsigned int ALL_DIGITS = (1u << (SIZE + 1)) - 2;
    //make a grid with our set size (always a square)
    int grid[SIZE][SIZE];
    // [A5-Claude] bookkeeping for the Sudoku rules. Bit d of rowUsed[r] is on
when digit d is already in row r;
    // colUsed and boxUsed do the same for columns and 3x3 boxes. A digit can
then be tested with one AND
    // instead of scanning 27 cells. The "= {}" starts every mask at zero (no
digits used yet)
    unsigned int rowUsed[SIZE] = {};
    unsigned int colUsed[SIZE] = {};
```

```
    unsigned int boxUsed[SIZE] = {};
    // [A5-Claude] turns true if two given digits break a Sudoku rule, which
means no solution can exist
    bool givensConflict = false;
    //make a filename
    std::string filename;
    // [A4+A5-Claude] change 1: one shared solutions.txt for every puzzle. main()
owns the file; each solver
    // holds a reference to it and writes its own section. Answers are written
the moment they are found
    std::ofstream& solutionFile;
    //make a counter for the solutions found
    //lucas edit, the solutionsFound should be zero
    //initialised when declared
    int solutionsFound = 0;

    // Checks if placing 'num' at grid[row][col] is valid

    bool isValid(int row, int col, int num) const {
        // [A4+A5-Claude] the old body scanned the row, column and box (27 cells)
on every try. Now it is a
        // single bit test: candidates() already knows which digits are legal
here, so shift that mask right by
        // 'num' and look at the lowest bit (1 means digit 'num' is legal in this
cell)
        return ((candidates(row, col) >> num) & 1u) != 0;
    }

    // [A5-Claude] Tells which of the nine 3x3 boxes a cell sits in. Boxes are
numbered 0..8, left to right and
    // then top to bottom, so the centre box is number 4. It picks the right
entry of boxUsed
    int boxOf(int row, int col) const {
        // row / 3 gives the band of boxes (0, 1 or 2); col / 3 gives the box
inside that band
        return (row / BLOCK_SIZE) * BLOCK_SIZE + (col / BLOCK_SIZE);
    }

    // [A5-Claude] Returns the digits that may still legally go into cell (row,
col), one bit per digit.
    // This is the only place where the row, column and box rules of Sudoku are
checked
    unsigned int candidates(int row, int col) const {
        // OR the three "already used" masks together: a 1 bit means that digit
is blocked for this cell
        unsigned int blocked = rowUsed[row] | colUsed[col] | boxUsed[boxOf(row,
col)];
        // keep only the digit bits (1..9) that are not blocked
        return ALL_DIGITS & ~blocked;
```

```
    }

    // [A5-Claude] Counts the 1 bits in a mask, i.e. how many candidate digits a
cell has left
    static int countBits(unsigned int mask) {
        int count = 0;          // running total of 1 bits seen so far
        while (mask != 0) {     // keep going until no 1 bits are left
            mask &= mask - 1;   // switch off the lowest 1 bit
            count++;            // and count it
        }
        return count;           // 0..9 for our masks
    }

    // [A5-Claude] Puts a digit into a cell and records it in the row, column and
box masks
    void place(int row, int col, int num) {
        const unsigned int bit = 1u << num;   // one-hot mask of this digit
(digit 5 -> binary 100000)
        grid[row][col] = num;                 // write the digit into the grid
        rowUsed[row] |= bit;                  // the digit is now used in this
row
        colUsed[col] |= bit;                  // ... in this column
        boxUsed[boxOf(row, col)] |= bit;      // ... and in this box
    }

    // [A5-Claude] Exact reverse of place(): empties the cell and frees its digit
in the three masks
    void unplace(int row, int col) {
        const unsigned int bit = 1u << grid[row][col];  // which digit is being
taken back out
        rowUsed[row] &= ~bit;                           // free it in this row
        colUsed[col] &= ~bit;                           // ... in this column
        boxUsed[boxOf(row, col)] &= ~bit;               // ... and in this box
        grid[row][col] = 0;                             // the cell is empty
again
    }

    // [A5-Claude] Runs once for every completed grid the search reaches. Keeping
this out of solveRecursive
    // means the search code only searches, and all the output work for a
solution lives in one place
    void recordSolution() {
        solutionsFound++;                                       // one more
solution
        std::cout << "\nSolution " << solutionsFound << ":\n";  // console label,
same as Assignment 4
        printGrid(std::cout);                                   // console copy
of the grid
        printGrid(solutionFile);                                // file copy,
```

```
same layout as the puzzle files
        solutionFile << "\n";                                   // blank line so
grids in the file stay apart
    }

    // Recursive helper to find all solutions using depth-first search with
backtracking
    void solveRecursive() {
    // gets the recursive solution
        // [A4+A5-Claude] no row/col arguments any more. Instead of walking the
grid left to right, every call
        // looks for the empty cell with the FEWEST legal digits and branches on
that one. A cell with one
        // option is a forced move and a cell with zero options is a dead end
that is spotted immediately,
        // so far fewer branches are tried than before
        int bestRow = -1;            // row of the most constrained empty cell
found so far (-1 = none found yet)
        int bestCol = -1;            // column of that same cell
        unsigned int bestMask = 0;   // that cell's legal digits, one bit per
digit
        int bestCount = SIZE + 1;    // how many legal digits it has; starts
above the maximum possible (9)

        // visit every cell once: cell numbers 0..80 map to row = cell / 9 and
col = cell % 9
        for (int cell = 0; cell < SIZE * SIZE; cell++) {
            int row = cell / SIZE;                          // row of this cell
            int col = cell % SIZE;                          // column of this cell
            if (grid[row][col] != 0) {                      // already filled (a
given or a placed digit)
                continue;                                   // nothing to decide
here, go to the next cell
            }
            unsigned int mask = candidates(row, col);      // the digits that
could still go into this cell
            int count = countBits(mask);                    // and how many of
them there are
            // A cell with zero candidates proves this branch cannot work.
            if (count == 0) {
                bestRow = row;
                bestCol = col;
                bestMask = 0;
                bestCount = 0;
                break;
            }

            // Otherwise, remember the cell with the fewest candidates.
            if (count < bestCount) {
```

```
                bestRow = row;
                bestCol = col;
                bestMask = mask;
                bestCount = count;
            }
        }


        // no empty cell was found, so the grid is full. Every digit went in
through the masks, so it is a
        // valid solution
        if (bestRow == -1) {
            recordSolution();       // count it, print it and save it
            return;                 // back out and keep looking for other
solutions
        }

        // Branch on the chosen cell, trying its legal digits in ascending order.
If it has no legal digits,
        // bestMask is 0, no digit passes the test below and the loop does
nothing, so a dead end simply ends
        // this branch with no special case
    //see canidates 1-9 on our current element
        for (int num = 1; num <= SIZE; num++) {
            if ((bestMask >> num) & 1u) {         // [A4+A5-Claude] is digit
'num' one of this cell's legal digits?
                place(bestRow, bestCol, num);     // put the digit down and
update the masks
                solveRecursive();                 // search the rest of the grid
                unplace(bestRow, bestCol);        // Backtrack: take it out again
and try the next digit
            }
        }
    }

public:
    //initialiser method, takes a sting and requires a filename
    //calculates the solutionsFound
    // [A4+A5-Claude] change 2: now receives the shared solutions.txt stream this
puzzle writes its section into
    SudokuSolver(const std::string& file, std::ofstream& out) : filename(file),
solutionFile(out), solutionsFound(0) {
        for (int row = 0; row < SIZE; row++) {
            for (int col = 0; col < SIZE; col++) {
        //zero intiialise all data
                grid[row][col] = 0;
            }
        }
    }
```

```
    // Reads the grid from the file
    bool loadPuzzle() {
    //make a file stream
        std::ifstream file(filename);
        if (!file.is_open()) {
            std::cerr << "Error: Could not open file " << filename << std::endl;
            return false;
        }
    //change -> intialised unitiliased zero
    //explicitly zero initialising this is a good idea
    //so i did it
        char cell = 0;

        for (int row = 0; row < SIZE; row++) {
            for (int col= 0; col < SIZE; col++) {
        //copies file data into cell char by char
                if (!(file >> cell)) {
            //means we can't pull any new chars which means we're short in
characters
                    std::cerr << "Error: Invalid or incomplete puzzle format in "
<< filename << std::endl;
                    return false;
                }
        //here we check to see the data return type
        //in this case, we're seeing if it's empty
        //or some char 1-9 or a bad char
                if (cell == '_') {
                    grid[row][col] = 0;
                } else if (cell >= '1' && cell <= '9') {
                    // [A4+A5-Claude] a given digit. If its row, column or box
already holds the same digit, two
                    // givens clash and no solution can exist. Either way the
digit is stored and added to the masks
                    int digit = cell - '0';              // the character '7'
becomes the number 7
                    if (!isValid(row, col, digit)) {     // this digit is already
used in the row/column/box
                        givensConflict = true;           // remember the clash
(checked later in solve())
                    }
                    place(row, col, digit);              // store the digit and
update the three masks
                } else {
                    std::cerr << "Error: Invalid character '" << cell << "' in "
<< filename << std::endl;
                    return false;
                }
            }
```

```
        }
        // [A5-Claude] all 81 cells are read. If another non-blank character is
still in the file it is not a
        // 9x9 puzzle (a 10th column would silently shift every row), so refuse
the file instead of ignoring it
        if (file >> cell) {
            std::cerr << "Error: Extra data after the 81st cell in " << filename
<< std::endl;
            return false;
        }
    //load puzzle returns a boolean, which is kinda weird but not
    //too bad of a choice. it makes sense to do the init method
        return true;
    }

    // Prints the grid
    // [A4+A5-Claude] the output stream is now a parameter, so one function
prints to the console and to the file
    void printGrid(std::ostream& out) const {
    //for all the rows, columns, display either
    //the empty _ char to represent a zero possibility
    //or displsy a number 1-9 to output what the value
    //ought to be when displaying
        for (int row= 0; row < SIZE; row++) {
            for (int col = 0; col < SIZE; col++) {
                if (grid[row][col] == 0) {
                    out << "_ ";
                } else {
                    out << grid[row][col] << " ";
                }
            }
            out << "\n";
        }
    }

    // Primary object method to run solving routine and output formatted results
    void solve() {
    //display the filename and the puzzle
        std::cout << "========================================\n";
        std::cout << "File: " << filename << "\n";
        std::cout << "----------------------------------------\n";
        std::cout << "Initial Puzzle:\n";
        printGrid(std::cout);
        // [A4+A5-Claude] changes 3 & 4: no per-puzzle open/close any more. Start
this puzzle's section in the
        // shared solutions.txt with its header line
        solutionFile << "Puzzle: " << filename << "\n";

    //here we print all solutions by just recursively
```

```
    //going and display stuff. solutionsFound is a public
    //var so we have space for it.
        solutionsFound = 0;
        // [A4+A5-Claude] if two givens clash there is nothing to search;
otherwise run the search
        if (givensConflict) {
            std::cout << "\nTwo of the given digits clash, so this puzzle has no
solution.\n";
        } else {
            solveRecursive();
        }


    //display solutions found as a either a no solutions
    //or as a count, actual printing is handled later
        if (solutionsFound == 0) {
            std::cout << "\nNo solution found\n";
            solutionFile << "No solution found.\n";                          //
[A5-Claude] file version: exactly this one line
        } else {
            std::cout << "\nSolutions found: " << solutionsFound << "\n";
            solutionFile << "Solutions found: " << solutionsFound << "\n";   //
[A5-Claude] file version: count line after all grids
        }
        solutionFile << "\n";   // [A4+A5-Claude] change 5: blank line between
this puzzle's section and the next
        std::cout << "========================================\n\n";
    }
};

int main() {
   ///get all the puzzles
    std::vector<std::string> puzzleFiles = {
        "puzzle1.txt",
        "puzzle2.txt",
        "puzzle3.txt",
        "puzzle4.txt",
        "puzzle5.txt"
    };
    // [A4+A5-Claude] change 6: one output file for every puzzle, opened once
before the loop
    std::ofstream solutionsFile("solutions.txt");
    if (!solutionsFile.is_open()) {
        std::cerr << "Error: Could not create solutions.txt\n";
        return 1;
    }
    //use dynamic iteration if we wanna add more puzzles
    for (const auto& filename : puzzleFiles) {
    //make a sudoku instance
```

```
        SudokuSolver solver(filename, solutionsFile);   // [A4+A5-Claude] every
solver writes into solutions.txt
    //can we load it?
        if (solver.loadPuzzle()) {
            solver.solve();//then solve it, which prints everything
        }
    }
    solutionsFile.close();
    //return the POSIX return code
    return 0;
}
```

Good note here: CoPilot refused to output it, because it read our conversation as HTML and got confused by symbols. I had to manually take its suggestions by hand, though luckily there were not many changes

Changes

Time Complexity

Theoretical

This, theoretically, changes nothing significant enough for it to be considered in O(n) time complexity. Therefore, I will leave my original theory as the basis. The same file writing is even happening, it's just the ouput and where it is changes

Actual

I reused the same benchmarking methodology as the Fox section above (`benchmark.py`), but pointed it at the three binaries relevant to this prompt instead: `fox.cpp` (the baseline, one `solutionN.txt` per puzzle) against `claude.cpp` and `copilot.cpp` (both of which now share one `solutions.txt` passed by reference). This is `benchmark_ai.py`:

```
#!/usr/bin/env python3
"""
Same methodology as benchmark.py, but compares the three single-file-output
variants from the Assignment 6 prompt instead of the Assignment 5 variants:
  - fox.cpp      original: one solutionN.txt per puzzle
  - claude.cpp   Claude Opus 5.5's rewrite: all puzzles share one solutions.txt
  - copilot.cpp  Microsoft Copilot's rewrite: all puzzles share one solutions.txt

The point is to check whether switching from N output streams to 1 shared
stream changed anything about how run time scales with search difficulty
(it shouldn't -- the change is only to I/O plumbing, not the search).

Usage:
    python3 benchmark_ai.py
"""

import sys
```

```
from pathlib import Path

from benchmark import (
    BLANK_COUNTS,
    PUZZLE5,
    ROOT,
    TIMEOUT_SECONDS,
    build_blank_order,
    fit_exponential_base,
    run_once,
    write_puzzle5,
)

SOLVERS = {
    "fox.cpp (per-puzzle solutionN.txt)": ROOT / "fox_cmp_bin",
    "claude.cpp (shared solutions.txt)": ROOT / "claude_bin",
    "copilot.cpp (shared solutions.txt)": ROOT / "copilot_bin",
}


def main():
    for binary in SOLVERS.values():
        if not binary.exists():
            print(f"error: {binary.name} not found, build it first")
            sys.exit(1)

    backup = PUZZLE5.read_bytes() if PUZZLE5.exists() else None
    blank_order = build_blank_order()

    results = {name: [] for name in SOLVERS}
    gave_up = set()
    name_w = max(len(n) for n in SOLVERS)

    try:
        for n_blanks in BLANK_COUNTS:
            write_puzzle5(blank_order, n_blanks)
            print(f"\n=== puzzle5.txt with {n_blanks} blank cells "
                  f"({81 - n_blanks} givens) ===")
            for name, binary in SOLVERS.items():
                if name in gave_up:
                    results[name].append(None)
                    print(f"  {name:{name_w}s}: skipped (already timed out "
                          f"at a smaller blank count)")
                    continue
                elapsed = run_once(binary)
                results[name].append(elapsed)
                if elapsed is None:
                    gave_up.add(name)
                    print(f"  {name:{name_w}s}: TIMEOUT (> {TIMEOUT_SECONDS:.0f}
```

```
s)")
                else:
                    print(f"  {name:{name_w}s}: {elapsed:9.4f} s")
    finally:
        if backup is not None:
            PUZZLE5.write_bytes(backup)
        else:
            PUZZLE5.unlink(missing_ok=True)

    col_w = max(len(n) for n in SOLVERS) + 2
    header = f"{'blanks':>7} | " + " | ".join(f"{n:>{col_w}s}" for n in SOLVERS)
    print("\n" + "=" * len(header))
    print(header)
    print("-" * len(header))
    for i, n_blanks in enumerate(BLANK_COUNTS):
        row = [f"{n_blanks:>7}"]
        for name in SOLVERS:
            t = results[name][i]
            row.append(f"{'timeout':>{col_w}s}" if t is None else
f"{t:>{col_w}.4f}")
        print(" | ".join(row))

    print("\nApproximate exponential growth base (time ~ base**blanks),")
    print("fit by linear regression of ln(time) vs blank count:")
    for name, times in results.items():
        base = fit_exponential_base(BLANK_COUNTS, times)
        if base is None:
            print(f"  {name:{name_w}s}: not enough non-timeout data points")
        else:
            print(f"  {name:{name_w}s}: base ~= {base:.3f}")


if __name__ == "__main__":
    main()
```

Running it twice (to make sure nothing was a fluke) gave:

```
================================================================================================================
 blanks |   fox.cpp (per-puzzle solutionN.txt) |    claude.cpp (shared
solutions.txt) |   copilot.cpp (shared solutions.txt)
----------------------------------------------------------------------------------------------------------------
      0 |                                0.1252 |
0.0054 |                               0.0053
     20 |                                0.0049 |
0.0048 |                               0.0048
     35 |                                0.0049 |
0.0048 |                               0.0051
     45 |                                0.0052 |
0.0049 |                               0.0049
     50 |                                0.0055 |
```

```
0.0049 |                                0.0048
     55 |                                 0.0203 |
0.0203 |                                0.0202
     58 |                                 0.1830 |
0.1845 |                                0.2561
     60 |                                 0.4488 |
0.3524 |                                0.4639
     61 |                                 7.9161 |
7.5221 |                                7.5998
     62 |                                timeout |
timeout |                               timeout
     63 |                                timeout |
timeout |                               timeout
     64 |                                timeout |
timeout |                               timeout
     65 |                                timeout |
timeout |                               timeout
     66 |                                timeout |
timeout |                               timeout
     68 |                                timeout |
timeout |                               timeout
     70 |                                timeout |
timeout |                               timeout
```

```
Approximate exponential growth base (time ~ base**blanks),
fit by linear regression of ln(time) vs blank count:
  fox.cpp (per-puzzle solutionN.txt): base ~= 1.044
  claude.cpp (shared solutions.txt) : base ~= 1.083
  copilot.cpp (shared solutions.txt): base ~= 1.086
```

As we can see, they stay the same. This is because I did not prompt Claude or Copilot to make any maintainability features, just prompted them to do the features asked of by the assignment. This will be done in improvement sections, by me, and by hand.

Otherwise, we can see no notable difference, confirming the time complexity theory not changing when we just do something small like switch our inputs

Space Complexity

Theoretical

There are no changes in theoretical space complexity, as space complexity is the space that the program uses while running, not the space of its output. Therefore, I also expect this to be the same

Actual

Same idea as the time measurement above: reuse the methodology, but swap the solvers out for `fox.cpp`, `claude.cpp`, and `copilot.cpp`. This is `benchmark_space_ai.py`, built on top of the existing `benchmark_space.py` helpers (`run_once_memory`, `fit_linear_slope`, `fmt_bytes`):

```
#!/usr/bin/env python3
"""
Same methodology as benchmark_space.py, but compares the three single-file-
output variants from the Assignment 6 prompt instead of the Assignment 5
variants:
  - fox.cpp      original: one solutionN.txt per puzzle (own ofstream member)
  - claude.cpp   Claude Opus 5.5's rewrite: all puzzles share one solutions.txt
                 (ofstream& reference member)
  - copilot.cpp  Microsoft Copilot's rewrite: all puzzles share one solutions.txt
                 (ofstream& reference member)

The point is to check whether going from N owned ofstream buffers to 1
shared-by-reference ofstream changed anything about peak memory use.

Usage:
    python3 benchmark_space_ai.py
"""

import sys

from benchmark import BLANK_COUNTS, PUZZLE5, ROOT, build_blank_order,
write_puzzle5
from benchmark_space import fit_linear_slope, fmt_bytes, run_once_memory

SOLVERS = {
    "fox.cpp (per-puzzle solutionN.txt)": ROOT / "fox_cmp_bin",
    "claude.cpp (shared solutions.txt)": ROOT / "claude_bin",
    "copilot.cpp (shared solutions.txt)": ROOT / "copilot_bin",
}


def main():
    for binary in SOLVERS.values():
        if not binary.exists():
            print(f"error: {binary.name} not found, build it first")
            sys.exit(1)

    backup = PUZZLE5.read_bytes() if PUZZLE5.exists() else None
    blank_order = build_blank_order()

    results = {name: [] for name in SOLVERS}
    gave_up = set()
    name_w = max(len(n) for n in SOLVERS)

    try:
        for n_blanks in BLANK_COUNTS:
            write_puzzle5(blank_order, n_blanks)
            print(f"\n=== puzzle5.txt with {n_blanks} blank cells "
                  f"({81 - n_blanks} givens) ===")
```

```
            for name, binary in SOLVERS.items():
                if name in gave_up:
                    results[name].append(None)
                    print(f"  {name:{name_w}s}: skipped (already timed out "
                          f"at a smaller blank count)")
                    continue
                peak = run_once_memory(binary)
                results[name].append(peak)
                if peak is None:
                    gave_up.add(name)
                    print(f"  {name:{name_w}s}: TIMEOUT")
                else:
                    print(f"  {name:{name_w}s}: {fmt_bytes(peak)} peak RSS")
    finally:
        if backup is not None:
            PUZZLE5.write_bytes(backup)
        else:
            PUZZLE5.unlink(missing_ok=True)

    col_w = max(len(n) for n in SOLVERS) + 2
    header = f"{'blanks':>7} | " + " | ".join(f"{n:>{col_w}s}" for n in SOLVERS)
    print("\n" + "=" * len(header))
    print(header)
    print("-" * len(header))
    for i, n_blanks in enumerate(BLANK_COUNTS):
        row = [f"{n_blanks:>7}"]
        for name in SOLVERS:
            v = results[name][i]
            cell = "timeout" if v is None else fmt_bytes(v)
            row.append(f"{cell:>{col_w}s}")
        print(" | ".join(row))

    print("\nLinear fit of peak RSS vs blank count (bytes added per extra")
    print("blank cell):")
    for name, values in results.items():
        slope = fit_linear_slope(BLANK_COUNTS, values)
        if slope is None:
            print(f"  {name:{name_w}s}: not enough non-timeout data points")
        else:
            print(f"  {name:{name_w}s}: {slope:+.1f} bytes/blank")


if __name__ == "__main__":
    main()
```

The results:

```
==========================================================================================
 blanks |   fox.cpp (per-puzzle solutionN.txt) |    claude.cpp (shared
solutions.txt) |   copilot.cpp (shared solutions.txt)
```

```
-------------------------------------------------------------------------------------------------
      0 |                              1.64 MiB |                              1.66
MiB |                             1.64 MiB
     20 |                              1.64 MiB |                              1.66
MiB |                             1.64 MiB
     35 |                              1.64 MiB |                              1.66
MiB |                             1.64 MiB
     45 |                              1.64 MiB |                              1.66
MiB |                             1.64 MiB
     50 |                              1.64 MiB |                              1.66
MiB |                             1.64 MiB
     55 |                              1.69 MiB |                              1.70
MiB |                             1.69 MiB
     58 |                              1.69 MiB |                              1.70
MiB |                             1.69 MiB
     60 |                              1.69 MiB |                              1.70
MiB |                             1.69 MiB
     61 |                              1.69 MiB |                              1.70
MiB |                             1.69 MiB
     62 |                               timeout |
timeout |                               timeout
     63 |                               timeout |
timeout |                               timeout
     64 |                               timeout |
timeout |                               timeout
     65 |                               timeout |
timeout |                               timeout
     66 |                               timeout |
timeout |                               timeout
     68 |                               timeout |
timeout |                               timeout
     70 |                               timeout |
timeout |                               timeout

Linear fit of peak RSS vs blank count (bytes added per extra
blank cell):
  fox.cpp (per-puzzle solutionN.txt): +895.6 bytes/blank
  claude.cpp (shared solutions.txt) : +895.6 bytes/blank
  copilot.cpp (shared solutions.txt): +895.6 bytes/blank
```

As you can see here too, the output becomes the exact same. There is no change that is noticable in space complexity between the two programs, and there is no change by implementing this feature into the space complexity.

Maintainability

All of the initial problems outside of the Assignment 6 problem statement, such as comments, inconsistency with the integers, magic numbers, etc, still remain. This was not what I prompted the AI on as it does not fit the project.

Both LLMs did slightly different ways of outputting that are worth noting in terms of maintainability. Not shockilignly, Opus did a much better job than CoPilot.

std::ofstream versus std::osstream

Confusingly, which is the C++ fashion, both of these have about the same name, and both of these have similar functionality. With std::ofstream, you can only pass in a file to use as a reference. It opens a C++ stream that allows you to write to the file, as per the lifetime of the reference.

In the other vein, std::osstream is the more advance version. std::osstream allows you to pretty much just write to anything, whether it be std::cout, std::ofstream, or other types of writeable data types. std::ofstream actually inherents from osstream. Therefore, the argument can be made that having the more general part helps maintainability, because to switch back to printing on screen the only change is changing the child object from a file to ouptput. Making more modular and dynamic code is defintely a plus.

Using Copilot

Microsoft Copilot, at least as I interacted with it, through the app, was awful. It couldn't give me a file version, like mentioned before, so I had to implement changes hand by hand. It assured me that it could generate a file output, and then failed to do so every time, hallucinating it's own capabilities. I've never wrote something like this before, but For future maintainability, I would not recommend using this LLM.

Comments

Just quickly mentioning that the comments clash even more with the new AI code, which is something I will reconcile in this just next upcoming section

Human Improvement

There isn't too much to improve on except for a couple small things, mostly because of the scope of this.

Changes

I chose Claude to do my improvements. This is because it used ofstream, which was better for maintainability. I also saw less additional self inserted comments which makes refactoring easier.

I can identify two major fixes to implement:

```
* Make it so that the output is only displayed to the file, and therefore speed
up the entire process of calculation
* Create a single, consistent file output in the given program
```

The first one is really easy, we just need to delete lines. For the second, I had Claude rephrase them using my explanation style, in the lower case with explcit explanations of what and how a feature is implemented for long blocks of code.

Output

Here's what Claude Opus 5.5 gave me after improving on the previous Claude file.

```
/*
    <@
    (KU/
     "
human.cpp - by google gemini flash
modified slightly by lucas frias
 Program:       EECS 348 Assignment 5 - Sudoku solver (Assignment5.cpp, revised
from human.cpp)
 Description:  Solves Sudoku problems by depth-first search with backtracking and
finds EVERY solution.
               Each solution is printed to the console and also saved to one
shared file, solutions.txt.
               Assignment 5 revision: digits are tracked in bitmasks, the empty
cell with the fewest
               legal digits is filled first, and contradictory givens are
detected while loading.
 Input:         puzzle1.txt ... puzzle5.txt (9 rows of 9 cells, '_' = empty cell,
1-9 = given digit)
 Output:        Console report for every puzzle, plus solutions.txt. For each
puzzle the file holds the line
               "Puzzle: x" (x = puzzle file name), then either all solution grids
(same layout as the puzzle
               files, one blank line after each grid) followed by the single line
"Solutions found: n", or just
               the single line "No solution found.". A blank line separates one
puzzle's section from the next
 Collaborators:  Google Gemini Flash, Lucas Frias, Claude (Anthropic)
 Other sources:  Gemini Flash wrote the Assignment 4 base; Claude (Anthropic)
wrote the Assignment 5 changes
 * Author:        Lucas Frias (by using Gemini Flash)
 * Created:       October 13th 2026
 * Revised:        2026-10-13, implemented fix features
 *                category to enum class Category, per Opus 5.5 pairing
 * Revised by:    Jacob Fox, 2026-10-26 (Assignment 5: solution files, faster
search, input checks)
 * Revised by:    Jacob Fox, 2026-11-10 (all puzzles now write to one
solutions.txt)
 * Build:         g++ Assignment5.cpp -std=c++11 -Wall -Wextra -o Assignment5.exe
 * Run:           ./Assignment5.exe (puzzle1.txt through puzzle5.txt must be in
the same folder)
*/
#include <iostream>
#include <fstream>
#include <vector>
#include <string>

class SudokuSolver {
//sudoku solver, our main class to solve sudoku
```

```
private:
    //make a private class with a size, blcok_size
    static const int SIZE = 9;
    static const int BLOCK_SIZE = 3;
    // all nine digits as one bitmask: bit d stands for digit d, so bits 1..9 are
on and bit 0 is
    // unused (0x3FE when SIZE is 9). sets of candidate digits are stored as
masks like this one
    static const unsigned int ALL_DIGITS = (1u << (SIZE + 1)) - 2;
    //make a grid with our set size (always a square)
    int grid[SIZE][SIZE];
    // bookkeeping for the sudoku rules. bit d of rowUsed[r] is on when digit d
is already in row r;
    // colUsed and boxUsed do the same for columns and 3x3 boxes. a digit can
then be tested with one and
    // instead of scanning 27 cells. the "= {}" starts every mask at zero (no
digits used yet)
    unsigned int rowUsed[SIZE] = {};
    unsigned int colUsed[SIZE] = {};
    unsigned int boxUsed[SIZE] = {};
    // turns true if two given digits break a sudoku rule, which means no
solution can exist
    bool givensConflict = false;
    //make a filename
    std::string filename;
    // the shared solutions.txt stream. main() opens it once and every solver
writes its own
    // section into it. answers are written the moment they are found, so no
solution is kept in memory
    std::ostream& solutionFile;
    //make a counter for the solutions found
    //lucas edit, the solutionsFound should be zero
    //initialised when declared
    int solutionsFound = 0;

    // checks if placing 'num' at grid[row][col] is valid

    bool isValid(int row, int col, int num) const {
        // the old body scanned the row, column and box (27 cells) on every try.
now it is a
        // single bit test: candidates() already knows which digits are legal
here, so shift that mask right by
        // 'num' and look at the lowest bit (1 means digit 'num' is legal in this
cell)
        return ((candidates(row, col) >> num) & 1u) != 0;
    }

    // tells which of the nine 3x3 boxes a cell sits in. boxes are numbered 0..8,
left to right and
```

```
    // then top to bottom, so the centre box is number 4. it picks the right
entry of boxUsed
    int boxOf(int row, int col) const {
        // row / 3 gives the band of boxes (0, 1 or 2); col / 3 gives the box
inside that band
        return (row / BLOCK_SIZE) * BLOCK_SIZE + (col / BLOCK_SIZE);
    }

    // returns the digits that may still legally go into cell (row, col), one bit
per digit.
    // this is the only place where the row, column and box rules of sudoku are
checked
    unsigned int candidates(int row, int col) const {
        // or the three "already used" masks together: a 1 bit means that digit
is blocked for this cell
        unsigned int blocked = rowUsed[row] | colUsed[col] | boxUsed[boxOf(row,
col)];
        // keep only the digit bits (1..9) that are not blocked
        return ALL_DIGITS & ~blocked;
    }

    // counts the 1 bits in a mask, i.e. how many candidate digits a cell has
left
    static int countBits(unsigned int mask) {
        int count = 0;          // running total of 1 bits seen so far
        while (mask != 0) {     // keep going until no 1 bits are left
            mask &= mask - 1;   // switch off the lowest 1 bit
            count++;            // and count it
        }
        return count;           // 0..9 for our masks
    }

    // puts a digit into a cell and records it in the row, column and box masks
    void place(int row, int col, int num) {
        const unsigned int bit = 1u << num;   // one-hot mask of this digit
(digit 5 -> binary 100000)
        grid[row][col] = num;                 // write the digit into the grid
        rowUsed[row] |= bit;                  // the digit is now used in this
row
        colUsed[col] |= bit;                  // ... in this column
        boxUsed[boxOf(row, col)] |= bit;      // ... and in this box
    }

    // exact reverse of place(): empties the cell and frees its digit in the
three masks
    void unplace(int row, int col) {
        const unsigned int bit = 1u << grid[row][col];  // which digit is being
taken back out
        rowUsed[row] &= ~bit;                           // free it in this row
```

```
        colUsed[col] &= ~bit;                           // ... in this column
        boxUsed[boxOf(row, col)] &= ~bit;               // ... and in this box
        grid[row][col] = 0;                             // the cell is empty
again
    }

    // runs once for every completed grid the search reaches. keeping this out of
solveRecursive
    // means the search code only searches, and all the output work for a
solution lives in one place.
    // solutions no longer print to std::cout here: a puzzle with many solutions
was spending most of its
    // time on console writes instead of search, since every grid went to the
terminal as well as the file.
    // writing only to solutionFile keeps the per-solution cost to the file write
    void recordSolution() {
        solutionsFound++;            // one more solution
        printGrid(solutionFile);     // file copy, same layout as the puzzle
files
        solutionFile << "\n";        // blank line so grids in the file stay
apart
    }

    // recursive helper to find all solutions using depth-first search with
backtracking
    void solveRecursive() {
    // gets the recursive solution
        // no row/col arguments any more. instead of walking the grid left to
right, every call
        // looks for the empty cell with the fewest legal digits and branches on
that one. a cell with one
        // option is a forced move and a cell with zero options is a dead end
that is spotted immediately,
        // so far fewer branches are tried than before
        int bestRow = -1;            // row of the most constrained empty cell
found so far (-1 = none found yet)
        int bestCol = -1;            // column of that same cell
        unsigned int bestMask = 0;   // that cell's legal digits, one bit per
digit
        int bestCount = SIZE + 1;    // how many legal digits it has; starts
above the maximum possible (9)

        // visit every cell once: cell numbers 0..80 map to row = cell / 9 and
col = cell % 9
        for (int cell = 0; cell < SIZE * SIZE; cell++) {
            int row = cell / SIZE;                          // row of this cell
            int col = cell % SIZE;                          // column of this cell
            if (grid[row][col] != 0) {                      // already filled (a
given or a placed digit)
```

```
                continue;                                   // nothing to decide
here, go to the next cell
            }
            unsigned int mask = candidates(row, col);      // the digits that
could still go into this cell
            int count = countBits(mask);                   // and how many of
them there are
            // a cell with zero candidates proves this branch cannot work.
            if (count == 0) {
                bestRow = row;
                bestCol = col;
                bestMask = 0;
                bestCount = 0;
                break;
            }

            // otherwise, remember the cell with the fewest candidates.
            if (count < bestCount) {
                bestRow = row;
                bestCol = col;
                bestMask = mask;
                bestCount = count;
            }
        }


        // no empty cell was found, so the grid is full. every digit went in
through the masks, so it is a
        // valid solution
        if (bestRow == -1) {
            recordSolution();       // count it, print it and save it
            return;                 // back out and keep looking for other
solutions
        }

        // branch on the chosen cell, trying its legal digits in ascending order.
if it has no legal digits,
        // bestMask is 0, no digit passes the test below and the loop does
nothing, so a dead end simply ends
        // this branch with no special case
    //see canidates 1-9 on our current element
        for (int num = 1; num <= SIZE; num++) {
            if ((bestMask >> num) & 1u) {         // is digit 'num' one of this
cell's legal digits?
                place(bestRow, bestCol, num);     // put the digit down and
update the masks
                solveRecursive();                 // search the rest of the grid
                unplace(bestRow, bestCol);        // backtrack: take it out again
and try the next digit
```

```
            }
        }
    }

public:
    //initialiser method, takes a sting and requires a filename
    //calculates the solutionsFound
    // now also receives the shared solutions.txt stream this puzzle writes its
section into
    SudokuSolver(const std::string& file, std::ostream& out) : filename(file),
solutionFile(out), solutionsFound(0) {
        for (int row = 0; row < SIZE; row++) {
            for (int col = 0; col < SIZE; col++) {
        //zero intiialise all data
                grid[row][col] = 0;
            }
        }
    }

    // reads the grid from the file
    bool loadPuzzle() {
    //make a file stream
        std::ifstream file(filename);
        if (!file.is_open()) {
            std::cerr << "Error: Could not open file " << filename << std::endl;
            return false;
        }
    //change -> intialised unitiliased zero
    //explicitly zero initialising this is a good idea
    //so i did it
        char cell = 0;

        for (int row = 0; row < SIZE; row++) {
            for (int col= 0; col < SIZE; col++) {
        //copies file data into cell char by char
                if (!(file >> cell)) {
            //means we can't pull any new chars which means we're short in
characters
                    std::cerr << "Error: Invalid or incomplete puzzle format in "
<< filename << std::endl;
                    return false;
                }
        //here we check to see the data return type
        //in this case, we're seeing if it's empty
        //or some char 1-9 or a bad char
                if (cell == '_') {
                    grid[row][col] = 0;
                } else if (cell >= '1' && cell <= '9') {
                    // a given digit. if its row, column or box already holds the
```

```
same digit, two
                    // givens clash and no solution can exist. either way the
digit is stored and added to the masks
                    int digit = cell - '0';              // the character '7'
becomes the number 7
                    if (!isValid(row, col, digit)) {     // this digit is already
used in the row/column/box
                        givensConflict = true;           // remember the clash
(checked later in solve())
                    }
                    place(row, col, digit);              // store the digit and
update the three masks
                } else {
                    std::cerr << "Error: Invalid character '" << cell << "' in "
<< filename << std::endl;
                    return false;
                }
            }
        }
        // all 81 cells are read. if another non-blank character is still in the
file it is not a
        // 9x9 puzzle (a 10th column would silently shift every row), so refuse
the file instead of ignoring it
        if (file >> cell) {
            std::cerr << "Error: Extra data after the 81st cell in " << filename
<< std::endl;
            return false;
        }
    //load puzzle returns a boolean, which is kinda weird but not
    //too bad of a choice. it makes sense to do the init method
        return true;
    }

    // prints the grid
    // the output stream is now a parameter, so one function prints to the
console and to the file
    void printGrid(std::ostream& out) const {
    //for all the rows, columns, display either
    //the empty _ char to represent a zero possibility
    //or displsy a number 1-9 to output what the value
    //ought to be when displaying
        for (int row= 0; row < SIZE; row++) {
            for (int col = 0; col < SIZE; col++) {
                if (grid[row][col] == 0) {
                    out << "_ ";
                } else {
                    out << grid[row][col] << " ";
                }
            }
```

```
            out << "\n";
        }
    }

    // writes the section for a puzzle that could not be loaded (missing file or
bad format), so
    // every listed puzzle still appears in solutions.txt. an unreadable puzzle
has no solution to report
    void recordLoadFailure() {
        solutionFile << "Puzzle: " << filename << "\n";
        solutionFile << "No solution found.\n\n";
    }

    // primary object method to run solving routine and output formatted results
    void solve() {
    //display the filename and the puzzle
        std::cout << "========================================\n";
        std::cout << "File: " << filename << "\n";
        std::cout << "----------------------------------------\n";
        std::cout << "Initial Puzzle:\n";
        printGrid(std::cout);
        // start this puzzle's section in solutions.txt with its header line
        solutionFile << "Puzzle: " << filename << "\n";

    //here we print all solutions by just recursively
    //going and display stuff. solutionsFound is a public
    //var so we have space for it.
        solutionsFound = 0;
        // if two givens clash there is nothing to search; otherwise run the
search
        if (givensConflict) {
            std::cout << "\nTwo of the given digits clash, so this puzzle has no
solution.\n";
        } else {
            solveRecursive();
        }


    //display solutions found as a either a no solutions
    //or as a count, actual printing is handled later
        if (solutionsFound == 0) {
            std::cout << "\nNo solution found\n";
            solutionFile << "No solution found.\n";                          //
file version: exactly this one line
        } else {
            std::cout << "\nSolutions found: " << solutionsFound << "\n";
            solutionFile << "Solutions found: " << solutionsFound << "\n";   //
file version: count line after all grids
        }
```

```
        solutionFile << "\n";   // blank line between this puzzle's section and
the next one
        std::cout << "========================================\n\n";
    }
};

int main() {
   ///get all the puzzles
    std::vector<std::string> puzzleFiles = {
        "puzzle1.txt",
        "puzzle2.txt",
        "puzzle3.txt",
        "puzzle4.txt",
        "puzzle5.txt"
    };
    // one output file for every puzzle, created (or emptied) once before the
loop
    std::ofstream solutionsFile("solutions.txt");
    if (!solutionsFile.is_open()) {
        std::cerr << "Error: Could not create output file solutions.txt" <<
std::endl;
        return 1;
    }
    //use dynamic iteration if we wanna add more puzzles
    for (const auto& filename : puzzleFiles) {
    //make a sudoku instance
        SudokuSolver solver(filename, solutionsFile);   // every solver writes
into solutions.txt
    //can we load it?
        if (solver.loadPuzzle()) {
            solver.solve();//then solve it, which prints everything
        } else {
            solver.recordLoadFailure();   // still give the puzzle its section in
solutions.txt
        }
    }
    // close the file (this also flushes it), then tell the user whether the save
worked
    solutionsFile.close();
    if (solutionsFile.fail()) {
        std::cerr << "Error: Could not write solutions.txt" << std::endl;
        return 1;
    }
    std::cout << "Saved to solutions.txt\n";
    //return the POSIX return code
    return 0;
}
```

All I did was make it so the output doesn't repeat itself twice, which vastly speeds up the algorithm to be faster than its unmodified self and my initial self too. The improvements keep the same output, but improve both the now optimized speed of the algorithm, and also now we got good and consistent documentation.

Reflection and Closing Thoughts

Thanks so much for another grade and thankfully the last Sudoku assignment. I've been getting a little tired of doing the same analysis with very little changes, so hopefully this is sufficient.

It was nice to collaborate with my peers, and I learned a lot from both Mr. Garcia and Mr. Fox. I hope that their LLMs came in use as well as their reviewing. I think the biggest struggle is confirming AI output. We can see hor Mr. Fox in this assignment, while Claude did improve this code, forgot that it didn't need to output the same thing twice, mostly because it was not instructed too. It's small things like this that really make or break development, which is why I think having strong metrics like time complexity just help that much more to verify these mechanical outputs.

Stay warm, Mr. Zhe. I hope you enjoyed this Assignment 6 and I'll catch you on the next one